
Sequences

UVM Sequences

Sequences in UVM

UVM Sequences provide an object-oriented mechanism to specify stimulus generation at the transaction level, which makes test writers more efficient and effective while also promoting abstraction and reuse. You will find an introduction to sequences in the Sequence Overview page.

Controlling Sequence Execution

Sequence execution is started by calling the `uvm_sequence start()` method, this associates the sequence with a sequencer and then calls the sequence's body method. Inside the sequence body method, other sequences can be executed or `sequence_items` can be generated and sent to the driver. Generally, stimulus generation is started with one main controlling sequence in the test class `run()` method, and this sequence then spawns other sequences which generate the stimulus within the testbench. Once a chain of sequences is started, the flow of execution can be hierarchical, or parallel, or executed in a randomized order. Sequences can also be randomized to change control or data variables. A library of sequences can also be created, but it is not recommended.

In order to handle the `sequence_items` arriving from the sequence, the driver has access to methods which are implemented in the sequencer, these give the driver several alternate means to indicate that a `sequence_item` has been consumed or to send responses back to the sequence.

The handling of `sequence_items` inside a sequence often relies to some extent on the how the driver processes sequence items. There are a number of common sequence-driver use models, which include:

- Unidirectional non-pipelined
- Bidirectional non-pipelined
- Pipelined

Warning:

Once a sequence has started execution it should be allowed to complete, if it is stopped prematurely, then there is a reasonable chance that the sequence-sequencer-driver communication mechanism will lock up.

Sequence Stimulus Generation Variations

Controlling Stimulus Generation On More Than One Driver

The sequence architecture of a driver, sequencer, `sequence_item` and sequence is orientated around a single interface. In most cases, a testbench will need to handle transactions on multiple interfaces and this is achieved through the use of the virtual sequence. A virtual sequence is a sequence that can start sub-sequences on multiple sequencers. The recommended way to implement a virtual sequence is for it to have handles for the target sequencers on which the sub-sequences run. The legacy approach is to use a virtual sequencer which is a normal sequencer which contains handles for the target sequencers.

Controlling Multiple Sequences Connected To A Single Driver

Multiple sequences can interact concurrently with a driver. The sequencer has an arbitration mechanism to ensure that only one sequence_item has access to the driver at any point in time. The choice of which sequence item is sent is dependent on an user selectable sequencer arbitration algorithm. There are five built in sequencer arbitration mechanisms and a hook for a user defined algorithm. A sequence can be started with a priority, this enables some of the arbitration mechanisms to give the correct order of precedence to multiple sequences running on a sequencer.

If responses are being returned from the driver to one of several sequences, the sequence id field in the sequence_item is used by the sequencer to route the response back to the right sequencer. The response handling code in the driver should use the set_id_info call to ensure that any response items have the same sequence id as their originating request.

In some cases, such as processing an interrupt, a sequence needs to gain exclusive access to the driver, in this case a sequence can use the grab or lock methods.

Layering

In many cases, sequences are used to generate streams of data objects which can be abstracted as layers, serial communication channels being one example and accessing a bus through a register indirection is another. The layering mechanism allows sequences to be layered on top of eachother, so that high level layer sequences are translated into lower level sequences transparently. This form of sequence generation allows complex stimulus to be built very rapidly.

Waiting For A Hardware Event

Whilst the driver takes care of normal hardware synchronisation, a sequence execution flow may need to be synchronised to a hardware event such as a transition on a sideband signal, or an end of reset condition. Rather than extend the driver and add another field to the sequence_item, it is possible to implement a wait_for_hardware_event method in a configuration object that contains a pointer to a virtual interface.

Interrupt Driven Stimulus

One variation of the waiting for a hardware event theme is to use interrupts to trigger the execution of sequences. This might result in the exclusive execution of an interrupt service routine, or it might emulate the control state machine for a hardware device that generates interrupts when it is ready or has completed a task.

Sequence Items

The UVM stimulus generation process is based on sequences controlling the behavior of drivers by generating sequence_items and sending them to the driver via a sequencer. The framework of the stimulus generation flow is built around the sequence structure for control, but the generation data flow uses sequence_items as the data objects.

As sequence_items are the foundation on which sequences are built, some care needs to be taken with their design. Sequence_item content is determined by the information that the driver needs in order to execute a pin level transaction; ease of generation of new data object content, usually by supporting constrained random generation; and other factors such analysis hooks.

Data Property Members

The content of the sequence_item is closely related to the needs of the driver. The driver relies on the content of the sequence_items it receives to determine which type of pin level transaction to execute. The sequence items property members will consist of data fields that represent the following types of information:

- Control - i.e. What type of transfer, what size
- Payload - i.e. The main data content of the transfer
- Configuration - i.e. Setting up a new mode of operation, error behavior etc
- Analysis - i.e. Convenience fields which aid analysis - time stamps, rolling checksums etc

Randomization Considerations

Sequence_items are randomized within sequences to generate traffic data objects. Therefore, stimulus data properties should generally be declared as rand, and the sequence_item should contain any constraints required to ensure that the values generated by default are legal, or are within sensible bounds. In a sequence, sequence_items are often randomized using in-line constraints which extend these base level constraints.

As sequence_items are used for both request and response traffic and a good convention to follow is that request properties should be rand, and that response properties should not be rand. This optimizes the randomization process and also ensures that any collected response information is not corrupted by any randomization that might take place.

For example consider the following bus protocol sequence_item:

```
class bus_seq_item extends uvm_sequence_item;

  // Request data properties are rand
  rand logic[31:0] addr;
  rand logic[31:0] write_data;
  rand bit read_not_write;
  rand int delay;

  // Response data properties are NOT rand
  bit error;
  logic[31:0] read_data;

  `uvm_object_utils (bus_seq_item)

function new(string name = "bus_seq_item");
  super.new(name);
endfunction
```

```
// Delay between bus cycles is in a sensible range
constraint at_least_1 { delay inside {[1:20]};}

// 32 bit aligned transfers
constraint align_32 {addr[1:0] == 0;}

// etc
endclass: bus_seq_item
```

Sequence Item Methods

The `uvm_sequence_item` inherits from the `uvm_object` via the `uvm_transaction` class. The `uvm_object` has a number of virtual methods which are used to implement common data object functions (copy, clone, compare, print, transaction recording) and these should be implemented to make the `sequence_item` more general purpose.

A `sequence_item` is often used in analysis traffic and it may be useful to add utility functions which aid functional coverage or analysis.

Transaction Methods

Introduction

When working with data object classes derived from `uvm_objects`, including ones derived from `uvm_transactions`, `uvm_sequence_items` and `uvm_sequences`, there are a number of methods which are defined for common operations on the data objects properties. In turn, each of these methods calls one or more virtual methods which are left for the user to implement according to the detail of the data members within the object. These methods and their corresponding virtual methods are summarized in the following table.

Method called by user	Virtual method	Purpose
copy	do_copy	Performs a deep copy of an object
clone	do_copy	Creates a new object and then does a deep copy of an object
compare	do_compare	Compares one object to another of the same type
convert2string	-	Returns a string representation of the object
print	do_print	Prints a the result of convert2string to the screen
sprint	do_print	Returns the result of convert2string
record	do_record	Handles transaction recording
pack	do_pack	Compresses object contents into a bit format
unpack	do_unpack	Converts a bit format into the data object format
to_struct	-	Optional: returns a struct representation of the object
from_struct	-	Optional: populates the data members of the object from the struct input

The `do_xxx` methods can be implemented and populated using ``uvm_field_xxx` macros, but the resultant code is inefficient, hard to debug and can be prone to error. The recommended approach is to implement the methods manually which will result in improvements in testbench performance and memory footprint. For more information on the issues involved see the page on macro cost benefit analysis.

Consider the following sequence_item which has properties in it that represent most of the common data types:

```
class bus_item extends uvm_sequence_item;

// Factory registration
`uvm_object_utils (bus_item)

// Properties - a selection of common types:
rand int delay;
rand logic[31:0] addr;
rand op_code_enum op_code;
string slave_name;
rand logic[31:0] data[];
bit response;

function new(string name = "bus_item");
    super.new(name);
endfunction

endclass: bus_item
```

The common methods that need to be populated for this sequence_item are:

do_copy

The purpose of the do_copy method is to provide a means of making a deep copy* of a data object. The do_copy method is either used on its own or via the uvm_objects clone() method which allows independent duplicates of a data object to be created. For the sequence_item example, the method would be implemented as follows:

```
// do_copy method:
function void do_copy(uvm_object rhs);

    bus_item rhs_;

    if(!$cast(rhs_, rhs)) begin
        uvm_report_error("do_copy:", "Cast failed");
        return;
    end
    super.do_copy(rhs); // Chain the copy with parent classes
    delay = rhs_.delay;
    addr = rhs_.addr;
    op_code = rhs_.op_code;
    slave_name = rhs_.slave_name;
    data = rhs_.data;
    response = rhs_.response;
endfunction: do_copy

// Example of how do_copy would be used:

// Directly:
```

```

bus_item A, B;

A.copy(B); // A becomes a deep copy of B

// Indirectly:
$cast(A, B.clone()); // Clone returns an uvm_object which needs
// to be cast to the actual type

```

Note that the rhs argument is of type `uvm_object` since it is a virtual method, and that it therefore needs to be cast to the actual transaction type before its fields can be copied. A design consideration for this method is that it may not always make sense to copy all property values from one object to another.

**A deep copy is one where the value of each of the individual properties in a data object are copied to another, as opposed to a shallow copy where just the data pointer is copied.*

do_compare

The `do_compare` method is called by the `uvm_object` `compare()` method and it is used to compare two data objects of the same type to determine whether their contents are equal. The `do_compare()` method should only be coded for those properties which can be compared.

The `uvm_comparer` policy object has to be passed to the `do_compare()` method for compatibility with the virtual method template, but it is not necessary to use it in the comparison function and performance can be improved by not using it.

```

// do_compare implementation:
function bit do_compare(uvm_object rhs, uvm_comparer comparer);
    bus_item rhs_;

    // If the cast fails, comparison has also failed
    // A check for null is not needed because that is done in the compare()
    // function which calls do_compare()
    if(!$cast(rhs_, rhs)) begin
        return 0;
    end
    return((super.do_compare(rhs, comparer) &&
        (delay == rhs_.delay) &&
        (addr == rhs_.addr) &&
        (op_code == rhs_.op_code) &&
        (slave_name == rhs_.slave_name) &&
        (data == rhs_.data) &&
        (response == rhs_.response)));
endfunction: do_compare

// Usage example - do_compare is not used directly
bus_item A, B;

if(!A.compare(B)) begin
    // Report and handle error
end
else begin

```

```
// Report and handle success
end
```

convert2string

In order to get debug or status information printed to a simulator transcript or a log file from a data object there needs to be a way to convert the objects contents into a string representation - this is the purpose of the convert2string() method. Calling the method will return a string detailing the values of each of the properties formatted for transcript display or for writing to a file. The format is determined by the user:

```
// Implementation example:
function string convert2string();
    string s;

    s = super.convert2string();
    // Note the use of \t (tab) and \n (newline) to format the data in
columns
    // The enumerated op_code types .name() method returns a string
corresponding to its value
    $sformat(s, "%s\n delay \t%0d\n addr \t%0h\n op_code \t%s\n slave_name
\t%s\n",
    s, delay, addr, op_code.name(), slave_name);
    // For an array we need to iterate through the values:
    foreach(data[i]) begin
        $sformat(s, "%s data[%0d] \t%0h\n", s, i, data[i]);
    end
    $sformat(s, "%s response \t%0b\n", s, response);
    return s;
endfunction: convert2string
```

do_print

The do_print() method is called by the uvm_object print() method. Its purpose is to print out a string representation of an uvm data object using one of the uvm_printer policy classes. The simplest way to implement the method is to set the printer string to the value returned by the convert2string() method.

```
function void do_print(uvm_printer printer);
    printer.m_string = convert2string();
endfunction: do_print
```

An alternative, higher performance version of this would use \$display() to print the value returned by convert2string(), but this would not allow use of the various features of the various uvm_printer policy classes for formatting the data.

```
function void do_print(uvm_printer printer);
    $display(convert2string());
endfunction: do_print
```

To achieve full optimization, avoid using the print() and sprint() methods all together and call the convert2string() method directly.

do_record

The `do_record()` method is intended to support the viewing of data objects as transactions in a waveform GUI. Like the printing data object methods, the principle is that the fields that are recorded are visible in the transaction viewer. The underlying implementation of the ``uvm_record_field` macro used in the `do_record()` method is simulator specific and for Questa involves the use of the `$add_attribute()` system call:

```
function void do_record(uvm_recorder recorder);
  super.do_record(recorder); // To record any inherited data members
  `uvm_record_field("delay", delay)
  `uvm_record_field("addr", addr)
  `uvm_record_field("op_code", op_code.name())
  `uvm_record_field("slave_name", slave_name)
  foreach(data[i]) begin
    `uvm_record_field($sformatf("data[%0d]", i), data[i])
  end
  `uvm_record_field("response", response)
endfunction: do_record
```

In order to get transaction viewing to work with Questa you need to:

- Implement the `do_record()` method as shown
- Set the recording_detail config item to UVM_FULL:

```
set_config_int("", "recording_detail", UVM_FULL);
```

The transactions that are recorded by implementing `do_record()` and by turning on the `recording_detail` are available in the sequencer with a transaction stream handle name of `aggregate_items`.

do_pack and do_unpack

These two methods are not commonly used and their purpose is to convert a data object into a bit stream (i.e. an integer) to allow a representation to be passed between language domains, for instance between SystemVerilog and C/C++. The recommended implementation of these two methods is likely to change with forthcoming versions of Questa and are not documented here. However, they are documented in the paper which can be found on the page macro cost benefit analysis.

to_struct and from_struct

The `to_struct()` and `from_struct()` methods are optionally inserted into the transaction to convert the data members of the object into a packed struct representation which would be suitable for synthesis into an emulator. The struct definition itself is contained with a separate package which is shared between the simulation and the emulation worlds. This package with the struct definition is imported into the agent's package where the transaction class is ``included`. The shared package is also imported in any BFM where the struct is used.

The `to_struct()` and `from_struct()` functions are not defined in the UVM base class library. That is why they don't follow the `do_*` convention that the other transaction functions follow.

The `to_struct()` function is called on the transaction object and returns a packed struct representation of the object. That packed struct can then be used directly to send information to an emulator.

```
function item_s to_struct();
  to_struct.addr    = addr;
  to_struct.data    = data;
```



```
to_struct.injerr = injerr;  
endfunction : to_struct
```

When information is received back from an emulator, it should be converted back to an object which can easily be passed around a UVM testbench. The `from_struct()` method performs that duty by populating the object with data from the packed struct.

```
function void from_struct(item_s item);  
  addr    = item.addr;  
  data    = item.data;  
  injerr  = item.injerr;  
endfunction : from_struct
```

Sequence API

Sequence API Fundamentals

A `uvm_sequence` is derived from an `uvm_sequence_item` and it is parameterized with the type of `sequence_item` that it will send to a driver via a sequencer. The two most important properties of a sequence are the body method and the `m_sequencer` handle.

The body Method

An `uvm_sequence` contains a task method called `body`. It is the content of the body method that determines what the sequence does.

The `m_sequencer` Handle

When a sequence is started it is associated with a sequencer. The `m_sequencer` handle contains the reference to the sequencer on which the sequence is running. The `m_sequencer` handle can be used to access configuration information and other resources in the UVM component hierarchy.

Running a sequence

To get a sequence to run there are three steps that need to occur:

Step 1 - Sequence Creation

Since the sequence is derived from an `uvm_object`, it is created using the factory:

```
my_sequence m_seq; // Declaration  
  
m_seq = my_sequence::type_id::create("m_seq");
```

Using the factory creation method allows the sequence to be overridden with a sequence of derived type as a means of varying the stimulus generation.

Step 2 - Sequence Configuration

The sequence may need to be configured - examples of configuration include:

- Setting up start values - e.g. a start address or data value
- Setting up generation loop variables - e.g. number of iterations, which index number to start from
- Setting up pointers to testbench resources - e.g. a register map

```
m_seq.no_iterations = 10; // Direct assignment of values

// Using randomization
if(!m_seq.randomize() with {no_iterations inside {[5:20]}}) begin
    `uvm_error("marker", "Randomization failure for m_seq")
end

// Assigning a test bench resource
m_seq.reg_map = env.reg_map;
```

Step 3 - Starting The Sequence

A sequence is started using a call to its start() method, passing as an argument a pointer to the sequencer through which it will be sending sequence_items to a driver. The start() method assigns the sequencer pointer to a sequencer handle called m_sequencer within the sequence and then calls the body task within the sequence. When the sequence body task completes, the start method returns. Since it requires the body task to finish and this requires interaction with a driver, start() is a blocking method.

The start method has three optional arguments which have defaults which means that most of the time they are not necessary:

```
virtual task start
    uvm_sequencer_base sequencer, // Pointer to sequencer
    uvm_sequence_base parent_sequence = null, // Relevant if called within
    a sequence
    integer this_priority = 100, // Priority on the bit sequencer
    bit call_pre_post = 1); // pre_body and post_body methods called

// For instance - called from an uvm_component - usually the test:
apb_write_seq.start(env.m_apb_agent.m_sequencer);

// Or called from within a sequence:
apb_compare_seq.start(m_sequencer, this);
```

It is possible to call the sequence start method without any arguments, and this will result in the sequence running without a direct means of being able to connect to a driver.

Sending a sequence_item to a driver

To send a sequence_item to a driver there are four steps that need to occur:

Step 1 - Creation

The sequence_item is derived from uvm_object and should be created via the factory:

Using the factory creation method allows the sequence_item to be overridden with a sequence_item of a derived type if required.

Step 2 - Ready - start_item()

The start_item() call is made, passing the sequence_item handle as an argument. This call blocks until the sequencer grants the sequence and the sequence_item access to the driver.

Step 3 - Set

The sequence_item is prepared for use, usually through randomization, but it may also be initialized by setting properties directly.

Step 4 - Go - finish_item()

The finish_item() call is made, which blocks until the driver has completed its side of the transfer protocol for the item. No simulation time should be consumed between start_item() and finish_item().

Step 5 - Response - get_response()

This step is optional, and is only used if the driver sends a response to indicate that it has completed transaction associated with the sequence_item. The get_response() call blocks until a response item is available from the sequencers response FIFO.

```
// Inside some sequence connected to a sequencer

my_sequence_item item;

task body;
    // Step 1 - Creation
    item = my_sequence_item::type_id::create("item");

    // Step 2 - Ready - start_item()
    start_item(item);
    // Step 3 - Set
    if(!item.randomize() with {address inside {[0:32'h4FFF_FFFF]};}) begin
        `uvm_error("body", "randomization failure for item")
    end
    // Step 4 - Go - finish_item()
    finish_item(item);
endtask: body
```

Late Randomization

In the sequence_item flow above, steps 2 and 3 could be done in any order. However, leaving the randomization of the sequence_item until just before the finish_item() method call has the potential to allow the sequence_item to be randomized according to conditions true at the time of generation. This is sometimes referred to as late randomization. The alternative approach is to generate the sequence_item before the start_item() call, in this case the item is generated before it is necessarily clear how it is going to be used.

In previous generation verification methodologies, such as Specman and the AVM, generation was done at the beginning of the simulation and a stream of pre-prepared sequence_items was sent across to the driver. With late randomization, sequence_items are generated just in time and on demand.

Coding Guidelines

Make sequence input variables rand, output variables non-rand

A good coding convention to follow is to make variables which are used as inputs by sequence random and to leave variables which are to collect output or response information as non-random. This ensures that a sequence can be configured by randomization with constraints. The same convention applies to sequence_items.

Obvious exceptions to this convention would be handles and strings within the sequence.

Do not use pre_/post_body in sequences

The pre_body() method is intended to be executed before the body method and the post_body() method is intended to be called after body has completed. In practice, the functionality that these two methods might contain can easily be put directly into the body method. If there is some common initialization work to be done, then this can be put into the body method of a base class and called using super.body().

The way in which sequences are started affects whether these methods are called or not. Using start_item(), finish_item() means that the methods are not called, and using start() means that they may or may not be called, depending on the arguments. It is therefore easier for the user if the pre_/post_body methods are not used.

Do not raise/drop objections from a sequence

It is much easier to coordinate the raising and dropping of objections from the test (see UVM Coding Guidelines), so objections should never be raised or lowered from a sequence. This rule extends to not using the set_automatic_phase_objection() method added in UVM1.2, since this causes the sequence to raise and lower an objection.

Use start for sequences, start_item, finish_item for sequence_items

Sequence_items should be sent to the driver using the start_item() and finish_item() calls. Sequences should be started using the start() call.

Justification:

Although sequences can also be started using start_item() and finish_item(), it is clearer to the user whether a sequence or a sequence item is being processed if this convention is followed.

Using the start() call also allows the user to control whether the sequences pre_body() and post_body() hook methods are called. By default, the start() method enables the calling of the pre and post_body() methods, but this can be disabled by passing an extra argument into the start() method. Also note that the start_item() and finish_item() calls do not call pre or post_body().

Using start() for sequences means that a user does not need to know whether a sub-sequence contains one of these hook methods.

Sequences should not directly consume time

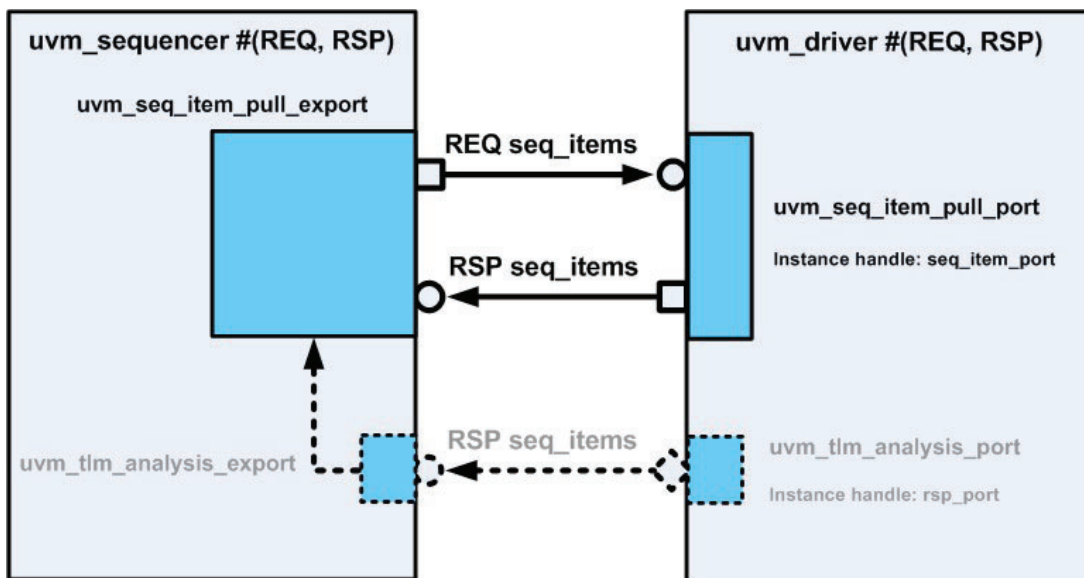
Sequence code should not explicitly consume time by using delay statements. They should only consume time by virtue of the process of sending `sequence_items` to a driver.

Justification:

Keeping this separation allows sequences to be reused as stimulus when an UVM testbench is linked to an emulator for hardware acceleration.

Sequencer

The transfer of request and response sequence items between sequences and their target driver is facilitated by a bidirectional TLM communication mechanism implemented in the sequencer. The `uvm_driver` class contains an `uvm_seq_item_pull_port` which should be connected to an `uvm_seq_item_pull_export` in the sequencer associated with the driver. The port and export classes are parameterized with the types of the `sequence_items` that are going to be used for request and response transactions. Once the port-export connection is made, the driver code can use the API implemented in the export to get request `sequence_items` from sequences and return responses to them.



Sequencer-Driver Connections

The connection between the driver port and the sequencer export is made using a TLM connect method during the connect phase:

```
// Driver parameterized with the same sequence_item for request &
response
// response defaults to request
class adpcm_driver extends uvm_driver #(adpcm_seq_item);
....
endclass: adpcm_driver

// Agent containing a driver and a sequencer - uninteresting bits left
out
class adpcm_agent extends uvm_agent;
```

```

adpcm_driver m_driver;
adpcm_agent_config m_cfg;
// uvm_sequencer parameterized with the adpcm_seq_item for request &
response
uvm_sequencer #(adpcm_seq_item) m_sequencer;

// Sequencer-Driver connection:
function void connect_phase(uvm_phase phase);
    if(m_cfg.active == UVM_ACTIVE) begin // The agent is actively driving
stimulus
        m_driver.seq_item_port.connect(m_sequencer.seq_item_export); // TLM
connection
        m_driver.vif = cfg.vif; // Virtual interface assignment
    end
endfunction: connect_phase

```

The connection between a driver and a sequencer is typically made in the `connect_phase()` method of an agent. With the standard UVM driver and sequencer base classes, the TLM connection between a driver and sequencer is a one to one connection - multiple drivers are not connected to a sequencer, nor are multiple sequencers connected to a driver. In addition to this bidirectional TLM port, there is an `analysis_port` in the driver which can be connected to an `analysis_export` in the sequencer to implement a unidirectional response communication path between the driver and the sequencer. This is a historical artifact and provides redundant functionality which is not generally used. The bidirectional TLM interface provides all the functionality required. If this analysis port is used, then the way to connect it is as follows:

```

// Same agent as in the previous bidirectional example:
class adpcm_agent extends uvm_agent;

adpcm_driver m_driver;
uvm_sequencer #(adpcm_seq_item) m_sequencer;
adpcm_agent_config m_cfg;

// Connect method:
function void connect_phase(uvm_phase phase );
    if(m_cfg.active == UVM_ACTIVE) begin
        m_driver.seq_item_port.connect(m_sequencer.seq_item_export); //
Always need this
        m_driver.rsp_port.connect(m_sequencer.rsp_export); // Response
analysis port connection
        m_driver.vif = cfg.vif;
    end
    //...
endfunction: connect_phase

endclass: adpcm_agent

```

Note that the bidirectional TLM connection will always have to be made to effect the communication of requests.

One possible use model for the `rsp_port` is to notify other components when a driver returns a response, otherwise it is not needed.

Driver-Sequence API

The `uvm_driver` is an extension of the `uvm_component` class that adds an `uvm_seq_item_pull_port` which is used to communicate with a sequence via a sequencer. The `uvm_driver` is a parameterized class and it is parameterized with the type of the request `sequence_item` and the type of the response `sequence_item`. In turn, these parameters are used to parameterize the `uvm_seq_item_pull_port`. The fact that the response `sequence_item` can be parameterized independently means that a driver can return a different response item type from the request type. In practice, most drivers use the same sequence item for both request and response, so in the source code the response `sequence_item` type defaults to the request `sequence_item` type.

The use model for the `uvm_driver` class is that it consumes request (REQ) `sequence_items` from the sequencers request FIFO using a handshake communication mechanism, and optionally returns response (RSP) `sequence_items` to the sequencers response FIFO. The handle for the `seq_item_pull_port` within the `uvm_driver` is the `seq_item_port`. The API used by driver code to interact with the sequencer is referenced by the `seq_item_port`, but is actually implemented in the sequencers `seq_item_export` (this is standard TLM practice).

UVM Driver API

The driver sequencer API calls are:

get_next_item

This method blocks until a REQ `sequence_item` is available in the sequencers request FIFO and then returns with a pointer to the REQ object.

The `get_next_item()` call implements half of the driver-sequencer protocol handshake, and it must be followed by an `item_done()` call which completes the handshake. Making another `get_next_item()` call before issuing an `item_done()` call will result in a protocol error and driver-sequencer deadlock.

try_next_item

This is a non-blocking variant of the `get_next_item()` method. It will return a null pointer if there is no REQ `sequence_item` available in the sequencers request FIFO. However, if there is a REQ `sequence_item` available it will complete the first half of the driver-sequencer handshake and must be followed by an `item_done()` call to complete the handshake.

item_done

The non-blocking `item_done()` method completes the driver-sequencer handshake and it should be called after a `get_next_item()` or a successful `try_next_item()` call.

If it is passed no argument or a null pointer it will complete the handshake without placing anything in the sequencer's response FIFO. If it is passed a pointer to a RSP `sequence_item` as an argument, then that pointer will be placed in the sequencer's response FIFO.

peek

If no REQ sequence_item is available in the sequencer's request FIFO, the peek() method will block until one is available and then return with a pointer to the REQ object, having executed the first half of the driver-sequencer handshake. Any further calls to peek() before a get() or an item_done() call will result in a pointer to the same REQ sequence item being returned.

get

The get() method blocks until a REQ sequence_item is available in the sequencer's request FIFO. Once one is available, it does a complete protocol handshake and returns with a pointer to the REQ object.

put

The put() method is non-blocking and is used to place a RSP sequence_item in the sequencer's response FIFO.

The put() method can be called at any time and is not connected with the driver-sequencer request handshaking mechanism.

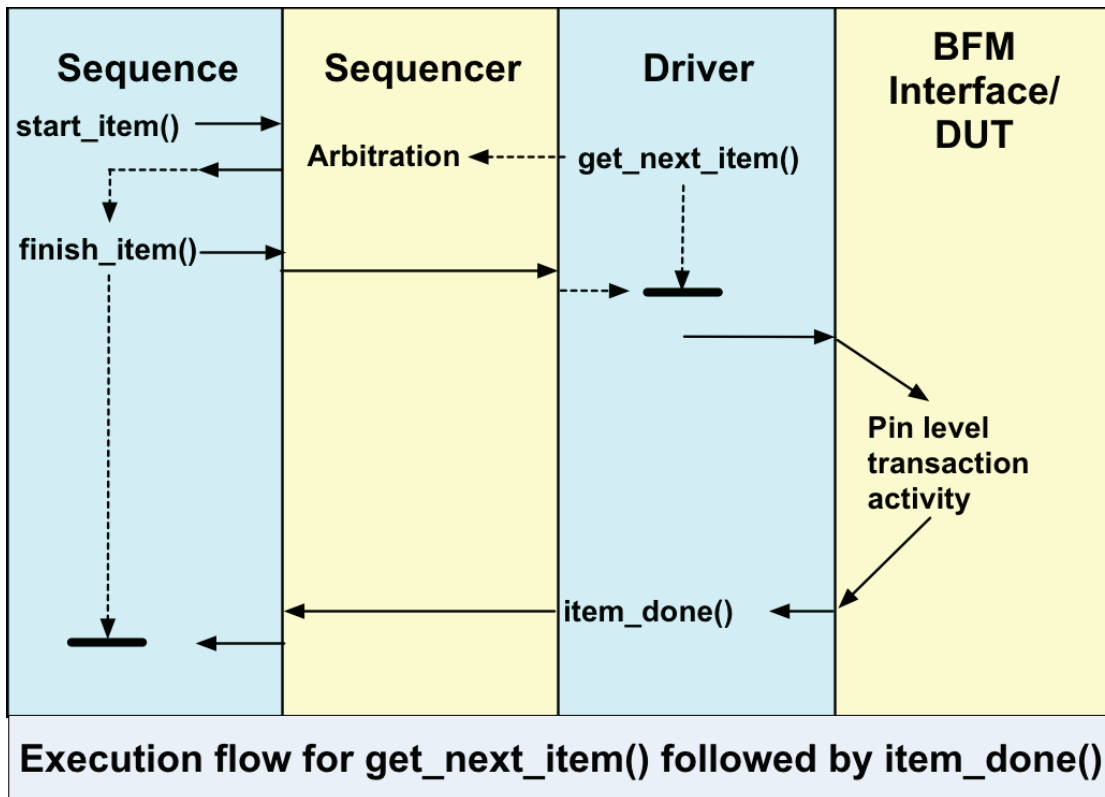
Note: The get_next_item(), get() and peek() methods initiate the sequencer arbitration process, which results in a sequence_item being returned from the active sequence which has selected. This means that the driver is effectively pulling sequence_items from the active sequences as it needs them.

Recommended Driver-Sequencer API Use Models

The purpose of the driver sequencer API is for the driver to receive a series of sequence_items from sequences containing the data required to initiate transactions, and for the driver to communicate back to the sequence that it has finished with the sequence_item and that it is ready for the next item. There are two common ways of doing this:

get_next_item() followed by item_done()

This use model allows the driver to get a sequence item from a sequence, process it and then pass a hand-shake back to the sequence using item_done(). No arguments should be passed in the item_done() call.



This is the preferred driver-sequencer API use model, since it provides a clean separation between the driver and the sequence.

```
//
// Driver run method
//
task run_phase( uvm_phase phase );
    bus_seq_item req_item;

    forever begin
        seq_item_port.get_next_item(req_item); // Blocking call returning
        the next transaction

        //BFM handles all pin wiggling and population of req_item with
        response data
        m_bfm.drive(req_item);

        seq_item_port.item_done(); // Signal to the sequence that the
        driver has finished with the item
    end
endtask: run
```

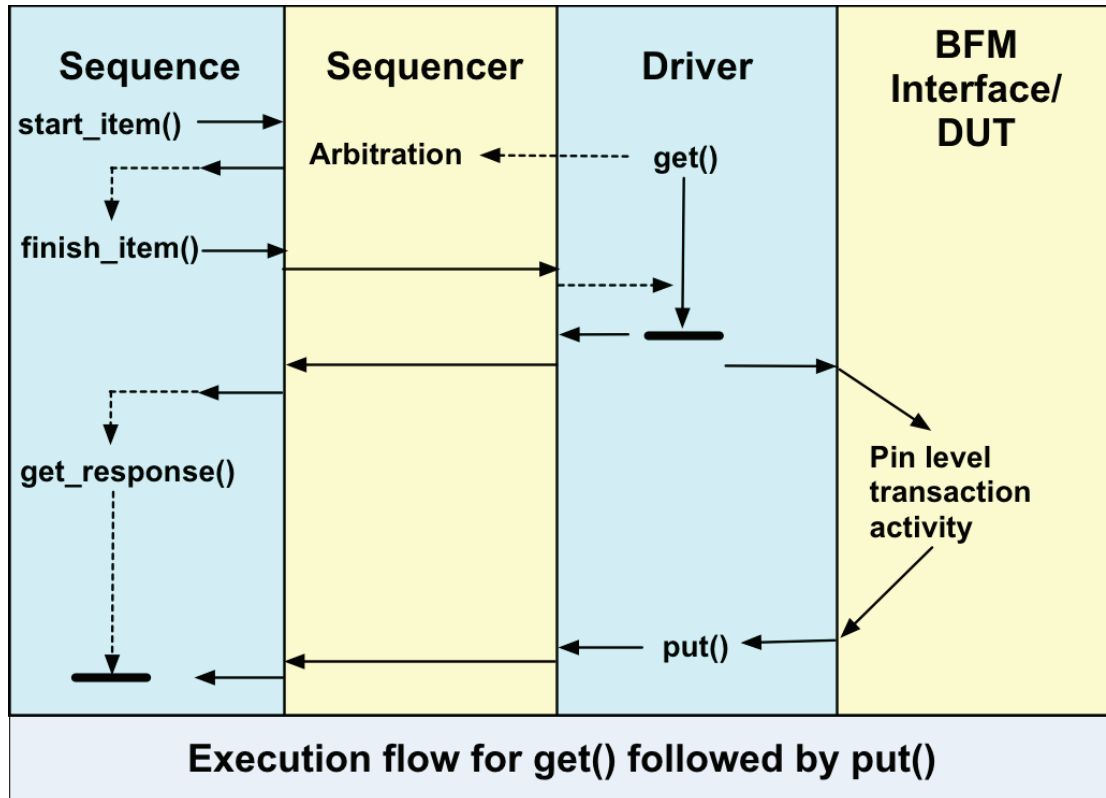
The corresponding sequence implementation would be a start_item() followed by a finish_item(). Since the driver proxy, driver BFM and the sequence are pointing to the same sequence_item, any data returning from the driver BFM can be referenced within the sequence via the sequence_item handle. In other words, when the handle to a sequence_item is passed as an argument to the finish_item() method the drivers get_next_item() method call completes with a pointer to the same sequence_item. When the driver makes any changes to the sequence_item it is

really updating the object inside the sequence. The drivers call to `item_done()` unblocks the `finish_item()` call in the sequence and then the sequence can access the fields in the `sequence_item`, including those which the driver may have updated as part of the response side of the pin level transaction.

```
//  
// Sequence body method:  
//  
task body();  
    bus_seq_item req_item;  
    bus_seq_item req_item_c;  
  
    req_item = bus_seq_item::type_id::create("req_item");  
  
    repeat(10) begin  
        $cast(req_item_c, req_item.clone); // Good practice to clone the  
req_item item  
        start_item(req_item_c);  
        req_item_c.randomize();  
        finish_item(req_item_c); // Driver has returned REQ with the  
response fields updated  
        `uvm_info("body", req_item_c.convert2string())  
    end  
endtask: body
```

get(req) followed by put(rsp)

With this use model, the driver does a `get(req)` which does the getting of the next `sequence_item` and sends back the sequence handshake in one go, before the driver has had any time to process the `sequence_item`. The driver uses the `put(rsp)` method to indicate that the `sequence_item` has been consumed and to return a response. The driver response handling should follow the guidelines given in the next section.



If this use model is followed, the sequence needs to follow the `finish_item()` call with a `get_response()` call which will block until the driver calls `put(rsp)`.

The drawbacks of this use model are that it is more complicated to implement on the driver side and that the sequence writer always has to remember to handle the response that is returned.

```
//
// run method within the driver
//
task run_phase( uvm_phase phase );
    REQ req_item; //REQ is parameterized type for requests
    RSP rsp_item; //RSP is parameterized type for responses

    bit [15:0] rdata;
    bit error;

    forever begin
        seq_item_port.get(req_item); // finish_item in sequence is
        unblocked

        //BFM handles all pin wiggling and returns response data as
        separate arguments
```

```

    m_bfm.drive(req_item, rdata, error);

    $cast(rsp_item, req_item.clone()); // Create a response transaction
    by cloning req_item
    rsp_item.set_id_info(req_item); // Set the rsp_item sequence id to
    match req_item
    if(req_item.read_or_write == READ) begin // Copy the bus data to
    the response fields
        rsp_item.read_data = rdata;
    end
    rsp_item.resp = error;
    seq_item_port.put(rsp_item); // Handshake back to the sequence via
    its get_response() call
    end
endtask

//
// Corresponding code within the sequence body method
//
task body();
    REQ req_item; //REQ is parameterized type for requests
    RSP rsp_item; //RSP is parameterized type for responses

    repeat(10) begin
        req_item = bus_seq_item::type_id::create("req_item");
        start_item(req_item);
        req_item.randomize();
        finish_item(req_item); // This passes to the driver get() call and
        is returned immediately
        get_response(rsp_item); // Block until a response is received
        `uvm_info("body", rsp_item.convert2string(), UVM_LOW);
    end
endtask: body

```

Routing response items back to the parent sequence

One of the complications of this use model arises when there are several sequences communicating with a driver through a sequencer. The sequencer looks after which request sequence_item gets routed to the driver from which sequence. However, when the driver creates a response sequence_item it needs to be routed back to the right sequence. The way in which the UVM takes care of this problem is that each sequence_item has a couple of id fields, one for the parent sequence and one to identify the sequence_item, these fields are used by the sequencer to route responses back to the parent sequence. A request sequence_item has these fields set by the sequencer as a result of the start_item() method, therefore, a new response sequence_item needs to take a copy of the requests id information so that it can be routed back to the originating sequence. The set_id_info() method is provided in the uvm_sequence_item base class for this purpose.

Coding Guidelines For Driver Response Items

set_id_info

The `uvm_sequence_item` contains an `id` field which is set by a sequencer during the `sequence_start_item()` call. This `id` allows the sequencer to keep track of which sequence each `sequence_item` is associated with, and this information is used to route response items back to the correct sequence. Although in the majority of cases only one sequence is actively communicating with a driver, the mechanism is always in use. The `sequence_item` `set_id_info` method is used to set a response item `id` from a the `id` field in request item.

If a request `sequence_item` is returned then the sequence `id` is already set. However, when a new or cloned response item is created, it must have its `id` set.

Use Clone For Pointer Safety

When a response item is sent back from a driver to a sequence, its pointer will be stored in the sequencer's response FIFO. If a response is not consumed before the next response pointer is sent then, unless the new response pointer is for a new object, both pointers will be referencing the same object. A common symptom of this problem is when successive reads of the FIFO yield objects with the same content.

The way to prevent this occurring is to clone the response item so that a new object is created and a pointer to this new object is passed to the sequencer response FIFO or to have different request and response types.

```
// Somewhere in a driver - request and response items
bus_seq_item req_item;
bus_seq_item rsp_item;

task run_phase( uvm_phase phase );

    forever begin
        seq_item_port.get(req_item);
        assert($cast(rsp_item, req_item.clone())); // This does not copy the
id info
        rsp_item.set_id_info(req_item); // This sets the rsp_item id to the
req_item id
        //
        // Do the pin level transaction, populate the response fields
        //
        // Return the response:
        seq_item_port.put(rsp_item);
        //
    end
endtask: run
```

Generating Stimulus with UVM Sequences

The `uvm_sequence_base` class extends the `uvm_sequence_item` class by adding a body task method. The sequence is used to generate stimulus through the execution of its body task. A sequence object is designed to be a transient dynamic object which means that it can be created, used and then garbage collected after it has been dereferenced.

The use of sequences in the UVM enables a very flexible approach to stimulus generation. Sequences are used to control the generation and flow of `sequence_items` into drivers, but they can also create and execute other sequences, either on the same driver or on a different one. Sequences can also mix the generation of `sequence_items` with the execution of sub-sequences. Since sequences are objects, the judicious use of polymorphism enables the creation of interesting randomized stimulus generation scenarios.

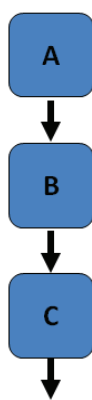
In any sequence stimulus generation process there are three primary layers in the flow:

1. **The master control thread** - This may be a run task in an UVM test component or a high level sequence such as a virtual sequence or a default sequence. The purpose of this thread is to start the next level of sequences.
2. **The individual sequences** - These may be stand-alone sequences that simply send `sequence_items` to a driver or they may in turn create and execute sub-sequences.
3. **The sequence_item** - This contains the information that enables a driver to perform a pin level transaction. The sequence item contains rand fields which are randomized with constraints during generation within a sequence.

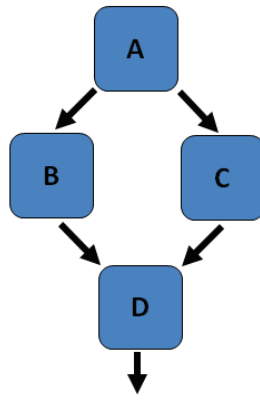
Sequence Execution Flow Alternatives

For any given sequence flow there are three basic models which may be mixed together to create different execution graphs.

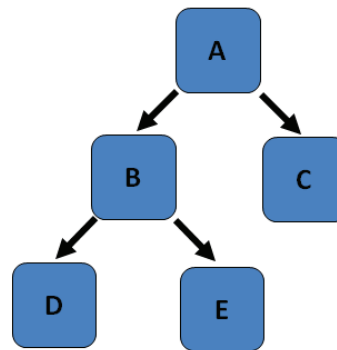
Different Generation Flow Options



Serial Execution:
`A.start(sqr);`
`B.start(sqr);`
`C.start(sqr);`



Parallel Execution:
`A.start(sqr);`
`fork`
 `B.start(sqr);`
 `C.start(sqr);`
`join`
`D.start(sqr);`



Hierarchical Execution:
`A.start(sqr);`
 // But this calls
 // B & C
 // And B calls D & E

Linear Execution Flow

With a linear flow, sequences are executed in order, with each sequence completing before the next one is started.

Parallel Execution Flow

In a parallel flow, fork-join is used to execute sequences in parallel. This means that two or more sequences may be interacting with a driver at any point in time. The SystemVerilog `join_any` and `join_none` constructs allow sequence processes to be spawned and left executing to overlap with subsequent sequences in the flow.

Coding Guideline - Allow fork-joined sequences to complete

A sequence must complete all its transfers before it is terminated, otherwise the sequencer handshake mechanism will be violated and the sequencer will lock up.

Using fork `join_any`/`join_nones` with sequences requires some care and attention to ensure that this rule is followed.

Do not fork, `join_any`, disable fork;

Using a fork <multiple_sequences> `join_any` followed by a disable fork in a sequence will result in the uncompleted sequence threads being terminated which will lock up the sequencer.

```
//
// DO NOT USE THIS PATTERN - Supplied as an example of what NOT to do
//
// Parent sequence body method running child sub_sequences within a
fork join_any
task body();
//
// Code creating and randomizing the child sequences
//
fork
    seq_A.start(m_sequencer);
    seq_B.start(m_sequencer);
    seq_C.start(m_sequencer);
join_any
// First past the post completes the fork and disables it
disable fork;
// Assuming seq_A completes first - seq_B and seq_C will be terminated
// in an indeterminate
// way, locking up the sequencer
```

Do not use fork `join_none` to exit a body method

Using fork, `join_none` around a body method to enable it to be executed in parallel with other sequences results in the sequences start method being terminated before the body method content has started communication with the sequencer. This coding style can be avoided by using fork `join_none` in the higher level control thread.

```
//
// DO NOT USE THIS PATTERN - Supplied as an example of what NOT to do
//
// Inside sequence_As body method
//
task body();
```

```

// Initialise etc
fork
    // The body code including other sequences and sequence_items
    // ...
    join_none
endtask body;
//
// The body task exits immediately, in the controlling thread the idea
is that sequence_A executes
// in parallel with other sequences:
//
task run_phase( uvm_phase phase );
// ....
    sequence_A_h.start(m_sequencer);
    another_seq_h.start(m_sequencer); // This sequence executes in
parallel with sequence_A_h
// ...

// The way to achieve the desired functionality is to remove the fork
join_none from sequence_A
// and to fork join the two sequences in the control thread:
//
task run;
// ....
    fork
        sequence_A_h.start(m_sequencer);
        another_seq_h.start(m_sequencer);
    join
// ....

```

Do not fork join a sequence which contains a forever loop

If a sequence contains a forever loop within its body method, it will loop indefinitely. If this sequence is started within a fork join of parent sequence and the parent sequence terminates, then the child sequences body method will continue to loop. If the parent sequence is terminated by a disable fork, then the child sequence may be caught mid-handshake and this will lock up the sequencer.

Hierarchical Execution Flow

A hierarchical flow starts with a top level sequence which creates and executes one or more sub-sequences, which in turn create and execute further sub-sequences. This approach is analogous to using layered software organised top-down so that a high level command is translated into a series of lower level calls until it reaches an atomic level at which bus level commands can be executed.

This layered hierarchical approach is described in the Hierarchy article.

Exploiting The Sequence As An Object

Sequences are objects which means that they have object characteristics which can be exploited during the stimulus generation process.

Randomized Fields

Like a `sequence_item`, a sequence can contain data fields that can be marked as rand fields. This means that a sequence can be made to behave differently by randomizing its variables before starting it. The use of constraints within the sequence allows the randomization to be within "legal" bounds, and the use of in-line constraints allows either a specific values or values within ranges to be generated.

Typically, the fields that are randomized within a sequence control the way in which it generates stimulus. For instance a sequence that moves data from one block of memory to another would contain a randomized start from address, a start to address and a transfer size. The transfer size could be constrained to be within a system limit - say 1K bytes. When the sequence is randomized, then the start locations would be constrained to be within the bounds of the relevant memory regions.

```
//
// This sequence shows how data members can be set to rand values
// to allow the sequence to either be randomized or set to a directed
// set of values in the controlling thread
//
// The sequence reads one block of memory (src_addr) into a buffer and
then
// writes the buffer into another block of memory (dst_addr). The size
// of the buffer is determined by the transfer size
//
class mem_trans_seq extends bus_seq_base;

`uvm_object_utils(mem_trans_seq)

// Randomised variables
rand logic[31:0] src_addr;
rand logic[31:0] dst_addr;
rand int transfer_size;

// Internal buffer
logic[31:0] buffer[];

// Legal limit on the page size is 1023 transfers
//
// No point in doing a transfer of 0 transfers
//
constraint page_size {
    transfer_size inside {[1:1024]};
}

// Addresses need to be aligned to 32 bit transfers
constraint address_alignment {
```

```

src_addr[1:0] == 0;
dst_addr[1:0] == 0;
}

function new(string name = "mem_trans_seq");
    super.new(name);
endfunction

task body;
    bus_seq_item req = bus_seq_item::type_id::create("req");
    logic[31:0] dst_start_addr = dst_addr; buffer =

    new[transfer_size];

    `uvm_info("run:", $sformatf("Transfer block of %0d words from %0h-%0h to %0h-%0h",
        transfer_size, src_addr, src_addr+((transfer_size-1)*4), dst_addr,
        dst_addr+((transfer_size-1)*4)), UVM_LOW)

    // Fill the buffer
    for(int i = 0; i < transfer_size-1; i++) begin
        start_item(req);
        if(!req.randomize() with {addr == src_addr; read_not_write == 1; delay < 3;}) begin
            `uvm_error("body", "randomization failed for req")
        end
        finish_item(req); buffer[i] =
        req.read_data;
        src_addr = src_addr + 4; // Increment to the next location
    end
    // Empty the buffer
    for(int i = 0; i < transfer_size-1; i++) begin
        start_item(req);
        if(!req.randomize() with {addr == dst_addr; read_not_write == 0; write_data == buffer[i]; delay
< 3;}) begin
            `uvm_error("body", "randomization failed for req")
        end
        finish_item(req);
        dst_addr = dst_addr + 4; // Increment to the next location
    end
    dst_addr = dst_start_addr;
    // Check the buffer transfer
    for(int i = 0; i < transfer_size-1; i++) begin
        start_item(req);
        if(!req.randomize() with {addr == dst_addr; read_not_write == 1; write_data == buffer[i]; delay
< 3;}) begin
            `uvm_error("body", "randomization failed for req")

```

```

    end
    finish_item(req);
    if(buffer[i] != req.read_data) begin
        `uvm_error("run:", $sformatf("Error in transfer @%0h : Expected
%0h, Actual %0h", dst_addr, buffer[i], req.read_data))
    end
    dst_addr = dst_addr + 4; // Increment to the next location
end
`uvm_info("run:", $sformatf("Finished transfer end addresses SRC: %0h DST:%0h",
                             src_addr, dst_addr), UVM_LOW)

endtask: body

endclass: mem_trans_seq

//
// This test shows how to randomize the memory_trans_seq
// to set it up for a block transfer
//
class seq_rand_test extends bus_test_base;

    `uvm_component_utils(seq_rand_test)

    function new(string name = "seq_rand_test", uvm_component parent =
null);
        super.new(name);
    endfunction

    task run_phase( uvm_phase phase ); phase.raise_objection( this , "start
mem_trans_seq" );
    mem_trans_seq seq = mem_trans_seq::type_id::create("seq");

    // Using randomization and constraints to set the initial values
    //
    // This could also be done directly
    //
    assert(seq.randomize() with {src_addr == 32'h0100_0800;
                                dst_addr inside
{[32'h0101_0000:32'h0103_0000]}};
                                transfer_size == 128;});

    seq.start(m_agent.m_sequencer);
    phase.drop_objection( this , "finished mem_trans_seq" );
endtask: run

endclass: seq_rand_test

```

A SystemVerilog class can randomize itself using `this.randomize()`, this means that a sequence can re-randomize itself in a loop.

Sequence Object Persistence

When a sequence is created and then executed using `sequence.start()`, the sequence's body method is executed. When the body method completes, the sequence object is still present in memory. This means that any information contained within the sequence and its object hierarchy is still accessible. This feature can be exploited to chain a series of sequences together, using the information from one sequence to seed the execution of another.

Taking the previous example of the memory transfer sequence, the same sequence could be re-executed without randomization to do a series of sequential transfers of the same size, before re-randomizing the sequence to do a transfer of a different size from a different start location.

Another example is a sequence that does a read or a block of reads from a peripheral. The next sequence can then use the content of the previous sequence's data fields to guide what it does.

```
//
// This class shows how to reuse the values persistent within a
sequence
// It runs the mem_trans_seq once with randomized values and then
repeats it
// several times without further randomization until the memory limit
is
// reached. This shows how the end address values are reused on each
repeat.
//
class rpt_mem_trans_seq extends bus_seq_base;

    `uvm_object_utils(rpt_mem_trans_seq)

    function new(string name = "rpt_mem_trans_seq");
        super.new(name);
    endfunction

    task body();
        mem_trans_seq trans_seq =
mem_trans_seq::type_id::create("trans_seq");

        // First transfer:
        assert(trans_seq.randomize() with {src_addr inside
{[32'h0100_0000:32'h0100_FFFF]};
                                dst_addr inside
{[32'h0103_0000:(32'h0104_0000 - (transfer_size*4))];
                                transfer_size < 512;
                                solve transfer_size before
dst_addr;});
        trans_seq.start(m_sequencer);
        // Continue with next block whilst we can complete within range
        // Each block transfer continues from where the last one left off
```

```

    while ((trans_seq.dst_addr + (trans_seq.transfer_size*4)) < 32'h0104_0000) begin
        trans_seq.start(m_sequencer);
    end

    endtask: body

endclass: rpt_mem_trans_seq

```

Exploiting Sequence Polymorphism

If a library of sequences is created, all of which are derived from the same object type, then it is possible to create these and put them into an array and then execute them in a random order. That order can be made random either by randomly generating the index to the array, or by shuffling the order of the array using the <array>.shuffle() method.

```

//
// This sequence executes some sub-sequences in a random order
//
class rand_order_seq extends bus_seq_base;

    `uvm_object_utils(rand_order_seq)

    function new(string name = "rand_order_seq");
        super.new(name);
    endfunction

    //
    // The sub-sequences are created and put into an array of
    // the common base type.
    //
    // Then the array order is shuffled before each sequence is
    // randomized and then executed
    //
    task body;
        bus_seq_base seq_array[4];

        seq_array[0] = n_m_rw_interleaved_seq::type_id::create("seq_0");
        seq_array[1] = rwr_seq::type_id::create("seq_1");
        seq_array[2] = n_m_rw_seq::type_id::create("seq_2");
        seq_array[3] = fill_memory_seq::type_id::create("seq_3");

        // Shuffle the array contents into a random order:
        seq_array.shuffle();
        // Execute all the array items in turn
        foreach(seq_array[i]) begin
            if(!seq_array[i].randomize()) begin
                `uvm_error("body", "randomization failed for req")
            end
            seq_array[i].start(m_sequencer);
        end
    end

```

```
endtask: body

endclass: rand_order_seq
```

Sequences can also be overridden with sequences of a derived type using the UVM factory, see the article on **overriding sequences** for more information. This approach allows a generation flow to change its characteristics without having to change the original sequence code.

Overriding Sequences and Sequence Items

Sometimes, during stimulus generation, it is useful to change the behavior of sequences or sequence items. The UVM factory provides an override mechanism to be able to substitute one object for another without changing any testbench code and without having to recompile it.

The UVM factory allows factory registered objects to be overridden by objects of a derived type. This means that when an object is constructed using the `<class_name>::type_id::create()` approach, then a change in the factory lookup for that object results in a pointer to an object of a derived type being returned. For instance, if there is sequence of type `seq_a`, and this is extended to create a sequence of type `seq_b` then `seq_b` can be used to override `seq_a`.

There are two types of factory override available - a type override, and an instance override.

Sequence Type Factory Override:

An type override means that any time a specific object type is constructed using the factory, a handle to the overridden type is returned. A type override can be used with a sequence, and it should be part of the test case configuration in the test. Once the type factory override is set, it will apply to all places in the subsequent sequence code where the overridden sequence object is constructed.

Sequence Instance Factory Override:

Specific sequences can be overridden via their "path" in the UVM testbench component hierarchy. For `uvm_components`, the path is defined as part of the build process via the name and parent arguments to the create method. However, sequences are `uvm_objects` and only use a name argument in their constructor and are not linked into the `uvm_component` hierarchy. The solution for creating a path for a sequence is to use two further arguments to the create method. The third argument passed to the sequence can be populated by the results of a `get_full_name()` call, or it can be any arbitrary string. The instance override then uses this string concatenated with the instance name field of the sequence to recreate the "instance path" of the sequence. For obvious reasons this means that a sequence instance override has to be pre-meditated as part of a sequences architecture.

```
//
// The build method of a test class:
//
// Inheritance:
//
// a_seq <- b_seq <- c_seq
//
function void build_phase( uvm_phase phase );
  m_env = sot_env::type_id::create("m_env", this);
```

```

// Set type override
b_seq::type_id::set_type_override(c_seq::get_type());
// Set instance override - Note the "path" argument see the line for
s_a creation
// in the run method
a_seq::type_id::set_inst_override(c_seq::get_type(), "bob.s_a");
endfunction: build

//
// Run method
//
task run_phase( uvm_phase phase );
  a_seq s_a; // Base type
  b_seq s_b; // b_seq extends a_seq
  c_seq s_c; // c_seq extends b_seq

  phase.raise_objection( this , "start a,b and c sequences" );

  // Instance name is "s_a" - first argument,
  // path name is "bob" but is more usually get_full_name() - third
argument
  s_a = a_seq::type_id::create("s_a",,"bob");
  // More usual create call
  s_b = b_seq::type_id::create("s_b");
  s_c = c_seq::type_id::create("s_c");

  s_a.start(m_env.m_a_agent.m_sequencer); // Results in c_seq being
executed
  s_b.start(m_env.m_a_agent.m_sequencer); // Results in c_seq being
executed
  s_c.start(m_env.m_a_agent.m_sequencer);

  phase.drop_objection( this , "a,b and c sequences done" );

endtask: run

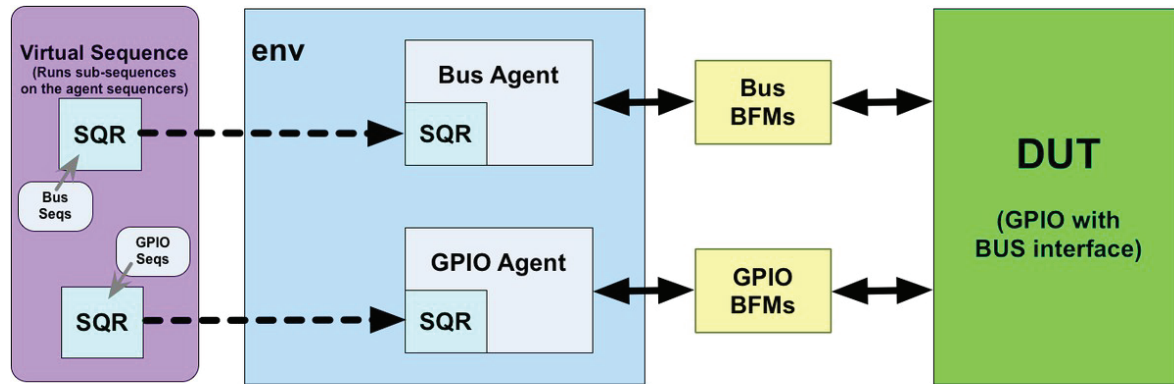
```

Sequence Item Overrides:

In principle, the same factory override mechanism could be used for `sequence_items`. However, it would require a driver to be written with prior knowledge of the derived type, so that it could cast a received `sequence_item` to the right type. Therefore, from a practical point of view, `sequence_items` are unlikely to be overridden.

Virtual Sequences

A virtual sequence is a sequence which controls stimulus generation using several sequencers. Since sequences, sequencers and drivers (proxy and BFM) are focused on point interfaces, almost all testbenches require a virtual sequence to coordinate the stimulus across different interfaces and the interactions between them. A virtual sequence is often the top level of the sequence hierarchy. A virtual sequence might also be referred to as a 'master sequence' or a 'coordinator sequence'.



The Virtual Sequence

A virtual sequence differs from a normal sequence in that its primary purpose is not to send sequence items. Instead, it generates and executes sequences on different target agents. To do this it contains handles for the target sequencers and these are used when the sequences are started.

```
// Creating a useful virtual sequence type:
typedef uvm_sequence #(uvm_sequence_item) uvm_virtual_sequence;

// Virtual sequence example:
class my_vseq extends uvm_virtual_sequence;
...
// Handles for the target sequencers:
a_sequencer_t a_sequencer;
b_sequencer_t b_sequencer;

task body();
...
// Start interface specific sequences on the appropriate target
sequencers:
    aseq.start( a_sequencer , this );
    bseq.start( b_sequencer , this );
endtask
endclass
```

In order for the virtual sequence to work, the sequencer handles have to be assigned. Typically, a virtual sequence is created in a test class in the run phase and the assignments to the sequencer handles within the virtual sequence object are made by the test. Once the sequencer handles are assigned, the virtual sequence is started using a null for the sequencer handle.

```
my_seq vseq = my_seq::type_id::create("vseq");
```



```
vseq.a_sequencer = env.subenv1.bus_agent.sequencer;
vseq.b_sequencer = env.subenv2.subsubenv1.bus_agent3.sequencer;

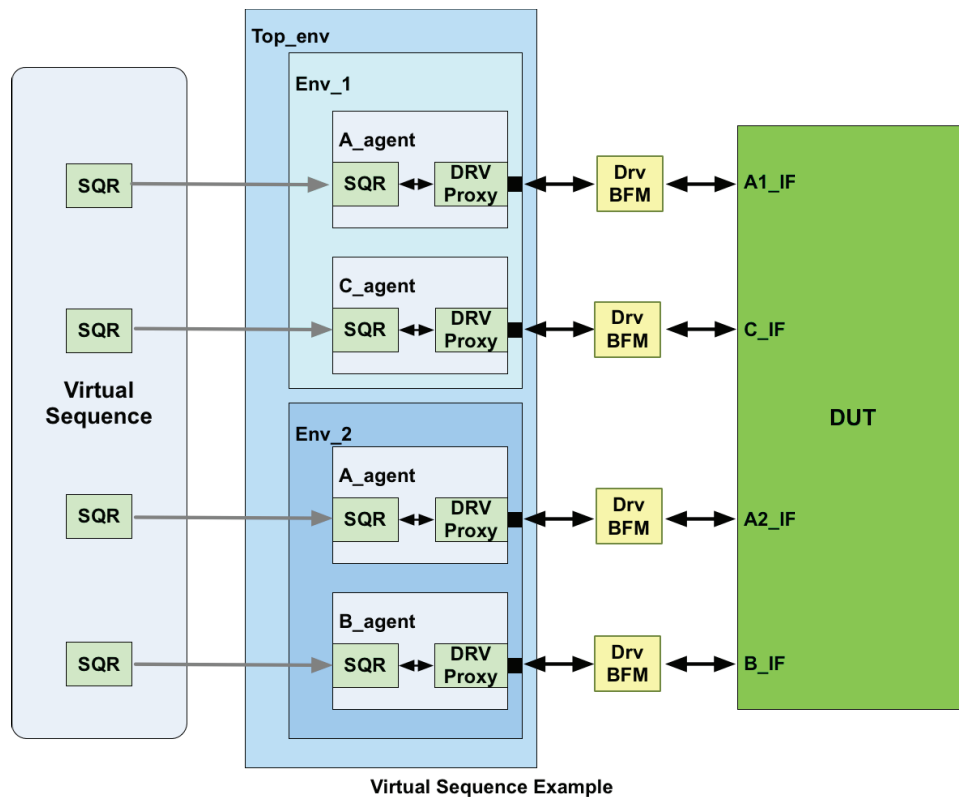
vseq.start( null );
```

There are several variations on the virtual sequence theme. There is nothing to stop the virtual sequence being started on a sequencer and sending sequence items to that sequencer whilst also executing other sequences on their target sequencers. The virtual sequence does not have to be executed by the test, it may be executed by an environment encapsulating a number of agents. For a large testbench with many agents and several areas of concern there may be several virtual sequences running concurrently.

In addition to target sequencer handles, a virtual sequence may also contain handles to other testbench resources such as register models which would be used by the sub-sequences.

Recommended Virtual Sequence Initialization Methodology

In order to use the UVM effectively, many organizations separate the implementation of the testbench from the implementation of the test cases. This is either a conceptual separation or an organizational separation. The testbench implementor should provide a test base class and a base virtual sequence class from which test cases can be derived. The test base class is responsible for building and configuring the verification environment component hierarchy, and specifying which virtual sequence(s) will run. The test base class should also contain a method for assigning sequence handles to virtual sequences derived from the virtual sequence base class. With several layers of vertical reuse, the hierarchical paths to target sequencers can become quite long. Since the hierarchical paths to the target sequencers are known to the testbench writer, this information can be encapsulated for all future test case writers.



As an example consider the testbench illustrated in the diagram. To illustrate a degree of virtual reuse, there are four target agents organised in two sub-environments within a top-level environment. The virtual sequence base class contains handles for each of the target sequencers:

```

class top_vseq_base extends uvm_sequence #(uvm_sequence_item);

`uvm_object_utils(top_vseq_base)

uvm_sequencer #(a_seq_item) A1;
uvm_sequencer #(a_seq_item) A2;
uvm_sequencer #(b_seq_item) B;
uvm_sequencer #(c_seq_item) C;

function new(string name = "top_vseq_base");
    super.new(name);
endfunction

endclass: top_vseq_base

```

In the test base class a method is created which can be used to assign the sequencer handles to the handles in classes derived from the virtual sequence base class.

```

class test_top_base extends uvm_test;

`uvm_component_utils(test_top_base)

env_top m_env;

function new(string name = "test_top_base", uvm_component parent =
null);
    super.new(name, parent);
endfunction

function void build_phase(uvm_phase phase);
    m_env = env_top::type_id::create("m_env", this);
endfunction: build_phase

// Method to initialize the virtual sequence handles
function void init_vseq(top_vseq_base vseq);
    vseq.A1 = m_env.m_env_1.m_agent_a.m_sequencer;
    vseq.C = m_env.m_env_1.m_agent_c.m_sequencer;
    vseq.A2 = m_env.m_env_2.m_agent_a.m_sequencer;
    vseq.B = m_env.m_env_2.m_agent_b.m_sequencer;
endfunction: init_vseq

endclass: test_top_base

```

In a test case derived from the test base class the virtual sequence initialization method is called before the virtual sequence is started.

```

class init_vseq_from_test extends test_top_base;

`uvm_component_utils(init_vseq_from_test)

```

```

function new(string name = "init_vseq_from_test", uvm_component parent
= null);
    super.new(name, parent);
endfunction

task run_phase(uvm_phase phase);
    vseq_A1_B_C vseq = vseq_A1_B_C::type_id::create("vseq");

    phase.raise_objection(this);

    init_vseq(vseq); // Using method from test base class to assign
sequence handles
    vseq.start(null); // null because no target sequencer

    phase.drop_objection(this);
endtask: run_phase

endclass: init_vseq_from_test

```

The virtual sequence is derived from the virtual sequence base class and requires no initialization code.

```

class vseq_A1_B_C extends top_vseq_base;

`uvm_object_utils(vseq_A1_B_C)

function new(string name = "vseq_A1_B_C");
    super.new(name);
endfunction

task body();
    a_seq a = a_seq::type_id::create("a");
    b_seq b = b_seq::type_id::create("b");
    c_seq c = c_seq::type_id::create("c");

    a.start(A1);
    fork
        b.start(B);
        c.start(C);
    join

endtask: body

endclass: vseq_A1_B_C

```

This example illustrates how the target sequencer handles can be assigned from the test case, but the same approach could be used for passing handles to other testbench resources such as register models and configuration objects which may be relevant to the operation of the virtual sequence or its sub-sequences.

Alternative Methods For Initializing A Virtual Sequence

In the preceding example, the virtual sequence is initialized from the test class, this is the recommended approach since it is so simple to implement. However, there are several alternative approaches that could be used:

- Putting the initialization method in the test package rather than making it a member of a test base class. This allows several variants of the init method to exist without having to define different base classes.
- Putting the initialization method in a mapping package which imports the env package(s) and the virtual sequence package. This separates out the initialization from the test. Several initialization methods could be defined for different virtual sequences. The mapping package would be imported into the test package.

```
package my_virtual_sequence_mapping_pkg;
//
// This package is specific to the test env and to the virtual sequence
//
import my_sequence_pkg::*;
import my_env_pkg::*;

function void init_my_virtual_sequence_from_my_env( my_virtual_sequence
vseq , my_env env );
    vseq.fabric_ports[0] = env.env1.a_agent.sequencer;
    vseq.fabric_ports[1] = env.env2.a_agent.sequencer;
    vseq.data_port = env.env1.b_agent.sequencer;
    vseq.control_port = env.env2.c_agent.sequencer;
end

// Other virtual sequence initialization methods could also be defined

endpackage
```

- Using the uvm_config_db to pass sequencer handles from the env to the virtual sequence. This can be made to work in small scaled environments, but may breakdown in larger scale environments, especially when multiple instantiations of the same env are used since there is no way to uniquify the look-up key for the sequencer in the uvm_config_db

```
// Inside the env containing the target sequencers:
//
function void connect_phase(uvm_phase phase);
//
    uvm_config_db #(a_sequencer)::set(null, "Sequencers", "a_sqr",
a_agent.m_sequencer);
    uvm_config_db #(b_sequencer)::set(null, "Sequencers", "b_sqr",
b_agent.m_sequencer);
//
endfunction

// Inside the virtual sequence base class:
//
a_sequencer A;
b_sequencer B;
```

```

// Get the sequencer handles back from the config_db
//
task body();
    if (!uvm_config_db #(a_sequencer)::get(null, "Sequencers", "a_sqr",
A)) begin
        `uvm_error("body", "a_sqr of type a_sequencer not found in the
uvm_config_db")
    end
    if (!uvm_config_db #(b_sequencer)::get(null, "Sequencers", "b_sqr",
B)) begin
        `uvm_error("body", "b_sqr of type b_sequencer not found in the
uvm_config_db")
    end
    // ....
endtask

```

- Using the find_all() method to find all the sequencers that match a search string in an environment. Again this relies on the sequencer paths being unique which is an assumption that will most likely break down in larger scale environments.

```

//
// A virtual sequence which runs stand-alone, but finds its own
sequencers
class virtual_sequence_base extends uvm_sequence #(uvm_sequence_item);

`uvm_object_utils(virtual_sequence)

// Sub-Sequencer handles
bus_sequencer_a A;
gpio_sequencer_b B;

// This task would be called as super.body by inheriting classes
task body;
    get_sequencers();
endtask: body

//
function void get_sequencers;
    uvm_component tmp[$];
    //find the A sequencer in the testbench
    tmp.delete(); //Make sure the queue is empty
    uvm_top.find_all("*m_bus_agent_h.m_sequencer_h", tmp);
    if (tmp.size() == 0)
        `uvm_fatal(report_id, "Failed to find mem sequencer")
    else if (tmp.size() > 1)
        `uvm_fatal(report_id, "Matched too many components when looking for
mem sequencer")

```

```
else
    $cast(A, tmp[0]);
    //find the B sequencer in the testbench
    tmp.delete(); //Make sure the queue is empty
    uvm_top.find_all("*m_gpio_agent_h.m_sequencer_h", tmp);
    if (tmp.size() == 0)
        `uvm_fatal(report_id, "Failed to find mem sequencer")
    else if (tmp.size() > 1)
        `uvm_fatal(report_id, "Matched too many components when looking for
mem sequencer")
    else
        $cast(B, tmp[0]);
endfunction: get_sequences

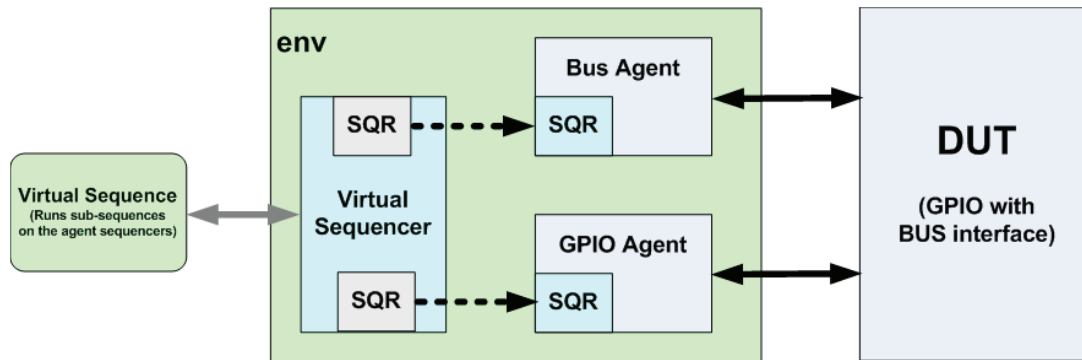
endclass: virtual_sequence_base
```

The Virtual Sequencer - An Alternative Methodology For Running Virtual Sequences

An alternative methodology for running virtual sequences is to use a virtual sequencer, which is a `uvm_sequencer` which contains the handles for the target sequencers. In this methodology, the virtual sequence is started on the virtual sequencer and it gets the handles for the target sequencers from the virtual sequencer. The limitation of this approach is that it is a fixed implementation which is very tightly coupled to the local hierarchy of an env and this adds complications with vertical reuse.

Virtual Sequencers (Not Recommended)

A virtual sequence is a sequence which controls a stimulus generation process using several sequencers. Since sequences, sequencers and drivers are focused on interfaces, almost all testbenches require a virtual sequence to coordinate the stimulus across different interfaces and the interactions between them.



The Virtual Sequencer and the Virtual Sequence

A virtual sequence can be implemented in one of two ways, the recommended way is to use a stand-alone virtual sequence and the 'legacy' alternative is to use virtual sequence that is designed to run on a virtual sequencer as described here.

A virtual sequencer is a sequencer that is not connected to a driver itself, but contains handles for sequencers in the testbench hierarchy.

Virtual sequences that run on virtual sequencers

A virtual sequence is designed to run on a virtual sequencer by adding code that gets the handles to the sub-sequencers from the virtual sequencer. The most convenient way to do this is to extend virtual sequences from a base class that does the sequencer handle assignment. The virtual sequencer is part of the UVM component hierarchy and so its sub-sequencer references can be assigned during the connect phase.

Typically, a virtual sequencer is inserted inside the env of a block level testbench, with the connect method of the block level env being used to assign the sub-sequencer handles. Since the virtual_sequence is likely to be in overall control of the simulation, it is usually created in the test run method and started on the virtual sequencer - i.e. `virtual_sequence.start(virtual_sequencer);`

A useful coding guideline is to give the sub-sequencers inside the virtual sequence a name that represents the interface that they are associated with, this makes life easier for the test writer. For instance, the sequencer for the master bus interface could be called "bus_master" rather than "master_axi_sequencer".

```
// Virtual sequencer class:
class virtual_sequencer extends uvm_sequencer #(uvm_sequence_item);

`uvm_component_utils(virtual_sequencer)

// Note that the handles are in terms that the test writer understands
bus_master_sequencer bus;
gpio_sequencer gpio;

function new(string name = "virtual_sequencer", uvm_component parent =
null);
    super.new(name, parent);
```

```

endfunction

endclass: virtual_sequencer

class env extends uvm_env;

// Relevant parts of the env which combines the
// virtual_sequencer and the bus and gpio agents
//
// Build:
function void build_phase( uvm_phase phase );
    m_bus_agent = bus_master_agent::type_id::create("m_bus_agent", this); m_gpio_agent =
    gpio_agent::type_id::create("m_gpio_agent", this); m_v_sqr =
    virtual_sequencer::type_id::create("m_v_sqr", this);
endfunction: build_phase

// Connect - where the virtual_sequencer is hooked up:
// Note that these references are constant in the context of this env
function void connect_phase( uvm_phase phase ); m_v_sqr.bus =
    m_bus_agent.m_sequencer; m_v_sqr.gpio =
    m_gpio_agent.m_sequencer;
endfunction: connect_phase

endclass: env

// Virtual sequence base class:
//
class virtual_sequence_base extends uvm_sequence #(uvm_sequence_item);

    `uvm_object_utils(virtual_sequence_base)

// This is needed to get to the sub-sequencers in the
// m_sequencer
    virtual_sequencer v_sqr;

// Local sub-sequencer handles
    bus_master_sequencer bus; gpio_sequencer
    gpio;

function new(string name = "virtual_sequence_base");
    super.new(name);
endfunction

// Assign pointers to the sub-sequences in the base body method:
task body();
    if (!$cast(v_sqr, m_sequencer)) begin
        `uvm_error(get_full_name(), "Virtual sequencer pointer cast

```



```

failed");
    end
    bus = v_sqr.bus; gpio =
    v_sqr.gpio;
endtask: body

endclass: virtual_sequence_base

// Virtual sequence class:
//
class example_virtual_seq extends virtual_sequence_base;

    random_bus_seq bus_seq;
    random_gpio_chunk_seq gpio_seq;

    `uvm_object_utils(example_virtual_seq)

    function new(string name = "example_virtual_seq");
        super.new(name);
    endfunction

    task body();
        super.body; // Sets up the sub-sequencer pointers
        gpio_seq = random_gpio_chunk_seq::type_id::create("gpio_seq"); bus_seq =
        random_bus_seq::type_id::create("bus_seq");

        repeat(20) begin
            bus_seq.start(bus);
            gpio_seq.start(gpio);
        end endtask:
    body

endclass: example_virtual_seq

// Inside the test class:
//
task run;
    example_virtual_sequence test_seq =
    example_virtual_sequencer::type_id::create("test_seq");

    //...
    test_seq.start(m_env.m_v_sqr);
    //...
endtask: run

```

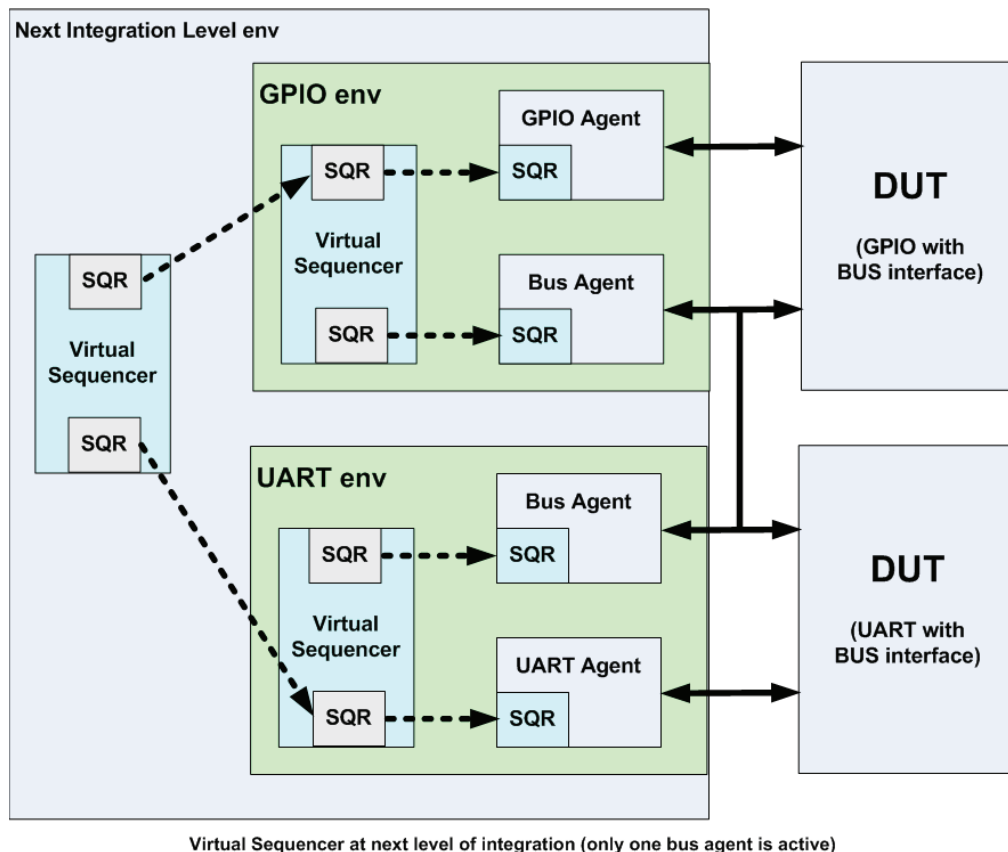
Chaining Virtual Sequencers for vertical reuse

When creating integrated verification environments that reuse block level envs, the virtual sequencers at the different levels of the testbench hierarchy can be chained together, allowing virtual sequences or sub-sequences to be run at any level. To achieve this level of flexibility, both the sub-sequencer handles and the virtual sequencer handles need to be encapsulated at each successive level.

```
// Virtual sequencer from the UART env
class uart_env_virtual_sqr extends uvm_sequencer #(uvm_sequence_item);
// ..
uart_sequencer uart;
bus_sequencer bus;
// ..
endclass: uart_env_virtual_sqr

// Virtual sequencer from the GPIO env
class gpio_env_virtual_sqr extends uvm_sequencer #(uvm_sequence_item);
// ..
gpio_sequencer gpio;
bus_sequencer bus;
// ..
endclass: gpio_env_virtual_sqr

// Virtual sequencer from the SoC env
class soc_env_virtual_sqr extends uvm_sequencer #(uvm_sequence_item);
//..
// Low level sequencers to support virtual sequences running across
```



```
// TB hierarchical boundaries
uart_sequencer uart;
bus_sequencer uart_bus;
gpio_sequencer gpio;
bus_sequencer gpio_bus;

// Virtual sequencers to support existing virtual sequences
//
uart_env_virtual_sqr uart_v_sqr;
gpio_env_virtual_sqr gpio_v_sqr;

// Low level sequencer pointer assignment:
// This has to be after connect because these connections are
// one down in the hierarchy
function void end_of_elaboration();
    uart = uart_v_sqr.uart;
    uart_bus = uart_v_sqr.bus;
    gpio = gpio_v_sqr.gpio;
    gpio_bus = gpio_v_sqr.bus;
endfunction: end_of_elaboration

endclass: soc_env_virtual_sqr
```

Coding Guideline: Virtual Sequences should check for null sequencer pointers before executing

Virtual sequence implementations are based on the assumption that they can run sequences on sub-sequencers within agents. However, agents may be passive or active depending on the configuration of the testbench. In order to prevent test cases crashing with null handle errors, virtual sequences should check that all of the sequencers that they intend to use have valid handles. If a null sequencer handle is detected, then they should bring the test case to an end with an `uvm\_fatal` call.

```
// Either inside the virtual sequence base class or in
// an extension of it that will use specific sequencers:
task body();
    if(!$cast(v_sqr, m_sequencer)) begin
        `uvm_error(get_full_name(), "Virtual sequencer pointer cast
failed")
    end
    if(v_sqr.gpio == null) begin
        `uvm_fatal(get_full_name(), "GPIO sub-sequencer null pointer: this
test case will fail, check config or virtual sequence")
    end
    else begin
        gpio = v_sqr.gpio;
    end
    if(v_sqr.gpio_bus == null) begin
        `uvm_fatal(get_full_name(), "BUS sub-sequencer null pointer: this
test case will fail, check config or virtual sequence")
    end
end
```

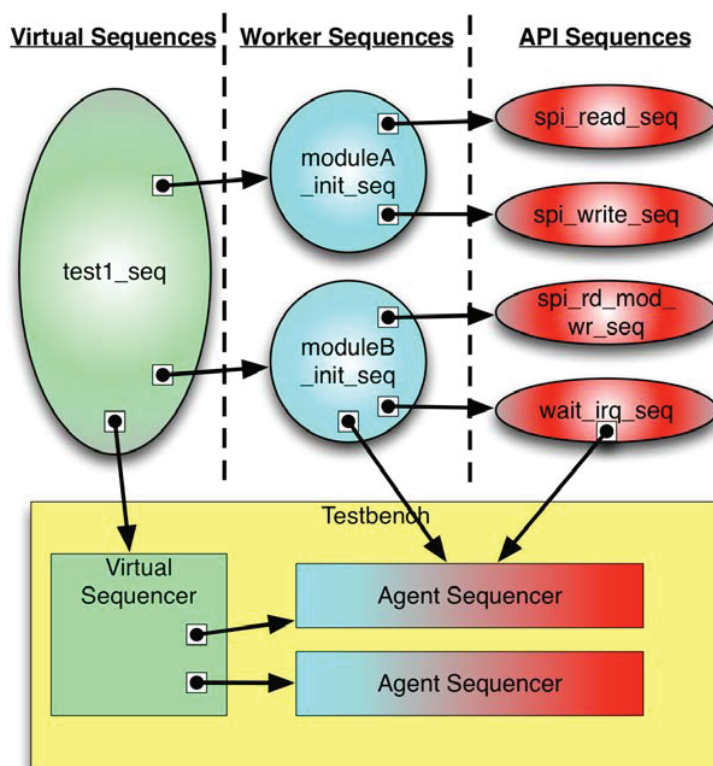
```

end
else begin
    gpio_bus = v_sqr.gpio_bus;
end
endtask: body

```

Hierarchical Sequences

When dealing with sequences, it helps to think in layers when considering the different functions that a testbench will be asked to perform. At the lowest layer associated with each agent are API sequences. The middle layer which makes use of the API sequences to get work done are worker sequences. Finally at the top of the testbench controlling everything is a virtual sequence which coordinates the running of worker sequences on the different target sequencers.



API Sequences

API sequences are the lowest layer in the sequence hierarchy. They also do the least amount of work in that they are doing a very targeted action when they are run. API sequences should perform small discreet actions like performing a read on a bus, a write on a bus, a read-modify-write on a bus or waiting for an interrupt or some other signal. The API sequences would be included in the SystemVerilog package which defines the Agent on which the sequences are intended to run. Two example API sequences would look as follows:

```

//API Sequence for doing a Read
class spi_read_seq extends uvm_sequence #(spi_item);
    `uvm_object_utils(spi_read_seq)

```

```

const string          report_id = "spi_read_seq";
rand bit [7:0]        addr;
bit [15:0]           rdata;

task body();
    req = spi_item::type_id::create("spi_request"); start_item(req);
    if ( !(req.randomize() with {req.addr == local::addr;} )) {
        `uvm_error(report_id, "Randomize Failed!") finish_item(req);
        rdata = req.data;
    }
endtask : body

task read(input bit [7:0] addr, output bit [15:0] read_data,
          input uvm_sequencer_base seqr, input uvm_sequence_base parent = null);
    this.addr = addr; this.start(seqr,
    parent); read_data = req.data;
endtask : read

endclass : spi_read_seq

//API Sequence for doing a Write
class spi_write_seq extends uvm_sequence #(spi_item);
    `uvm_object_utils(spi_write_seq)

    const string          report_id = "spi_write_seq";
    rand bit [7:0]        addr;
    rand bit [15:0]       wdata;

    task body();
        req = spi_item::type_id::create("spi_request"); start_item(req);
        if ( !(req.randomize() with {req.addr == local::addr;
                                   req.data == local::wdata; } )) {
            `uvm_error(report_id, "Randomize Failed!") finish_item(req);
        }
    endtask : body

    task write(bit [7:0] addr, bit [15:0] write_data,
             uvm_sequencer_base seqr, uvm_sequence_base parent = null); this.addr = addr;
        this.wdata = write_data;
        this.start(seqr, parent);
    endtask : write

```

```
endclass : spi_write_seq
```

Worker Sequences

Worker sequences make use of the low level API sequences to build up middle level sequences. These mid-level sequences could do things such as dut configuration, loading a memory, etc. Usually a worker sequence would only be sending sequence items to a single sequencer. A worker sequence would look like this:

```
//Worker sequence for doing initial configuration for Module A
class moduleA_init_seq extends uvm_sequence #(spi_item);
    `uvm_object_utils(moduleA_init_seq)

    const string      report_id = "moduleA_init_seq";
    spi_read_seq      read;
    spi_write_seq      write;

    task body();
        read = spi_read_seq::type_id::create("read");
        write = spi_write_seq::type_id::create("write");

        //Configure registers in Module
        //Calling start
        write.addr = 8'h20;
        write.wdata = 16'h00ff;
        write.start(m_sequencer, this);

        //Using the write task
        write.write(8'h22, 16'h0100, m_sequencer, this);

        //Other register writes

        //Check that Module A is ready
        read.addr = 8'h2c;
        read.start(m_sequencer, this);
        if (read.rdata != 16'h0001)
            `uvm_fatal(report_id, "Module A is not ready")

    endtask : body
endclass : moduleA_init_seq
```

Virtual Sequences

Virtual sequences are used to call and coordinate all the worker sequences. In most cases, designs will need to be initialized before random data can be sent in. The virtual sequence can call the worker initialization sequences and then call other worker sequences or even API sequences if it needs to do a low level action. The virtual sequence will either contain handles to the target sequencers (recommended) or be running on a virtual sequencer which allows access to all of the sequencers that are needed to run the worker and API sequences. An example virtual sequence

would look like this:

```
//Virtual Sequence controlling everything
class test1_seq extends uvm_sequence #(uvm_sequence_item);
  `uvm_object_utils(test1_seq)

  const string          report_id = "test1_seq";

  // These handles will be assigned by an init method in the test
  uvm_sequencer_base    spi_seqr;
  uvm_sequencer_base    modA_seqr;
  uvm_sequencer_base    modB_seqr;

  moduleA_init_seq      modA_init;
  moduleB_init_seq      modB_init;

  moduleA_rand_data_seq  modA_rand_data;
  moduleB_rand_data_seq  modB_rand_data;

  spi_read_seq          spi_read;
  bit [15:0]            read_data;

  task body();

    modA_init = moduleA_init_seq::type_id::create("modA_init"); modB_init =
    moduleB_init_seq::type_id::create("modB_init");

    modA_rand_data =
    moduleA_rand_data_seq::type_id::create("modA_rand_data");
    modB_rand_data =
    moduleB_rand_data_seq::type_id::create("modB_rand_data");

    spi_read = spi_read_seq::type_id::create("spi_read");

    //Do Initial Config
    fork
      modA_init.start(spi_seqr, this);
      modB_init.start(spi_seqr, this);
    join

    //Now start random data (These would probably be started on
different sequencers for a real design) fork
      modA_rand_data.start(modA_seqr, this);
      modB_rand_data.start(modB_seqr, this);
    join

    //Do a single read to check completion
    spi_read.read(8'h7C, read_data, spi_seqr, this);
```

```

if (read_data != 16'hffff)
    `uvm_error(report_id, "Test Failed!")

endtask : body

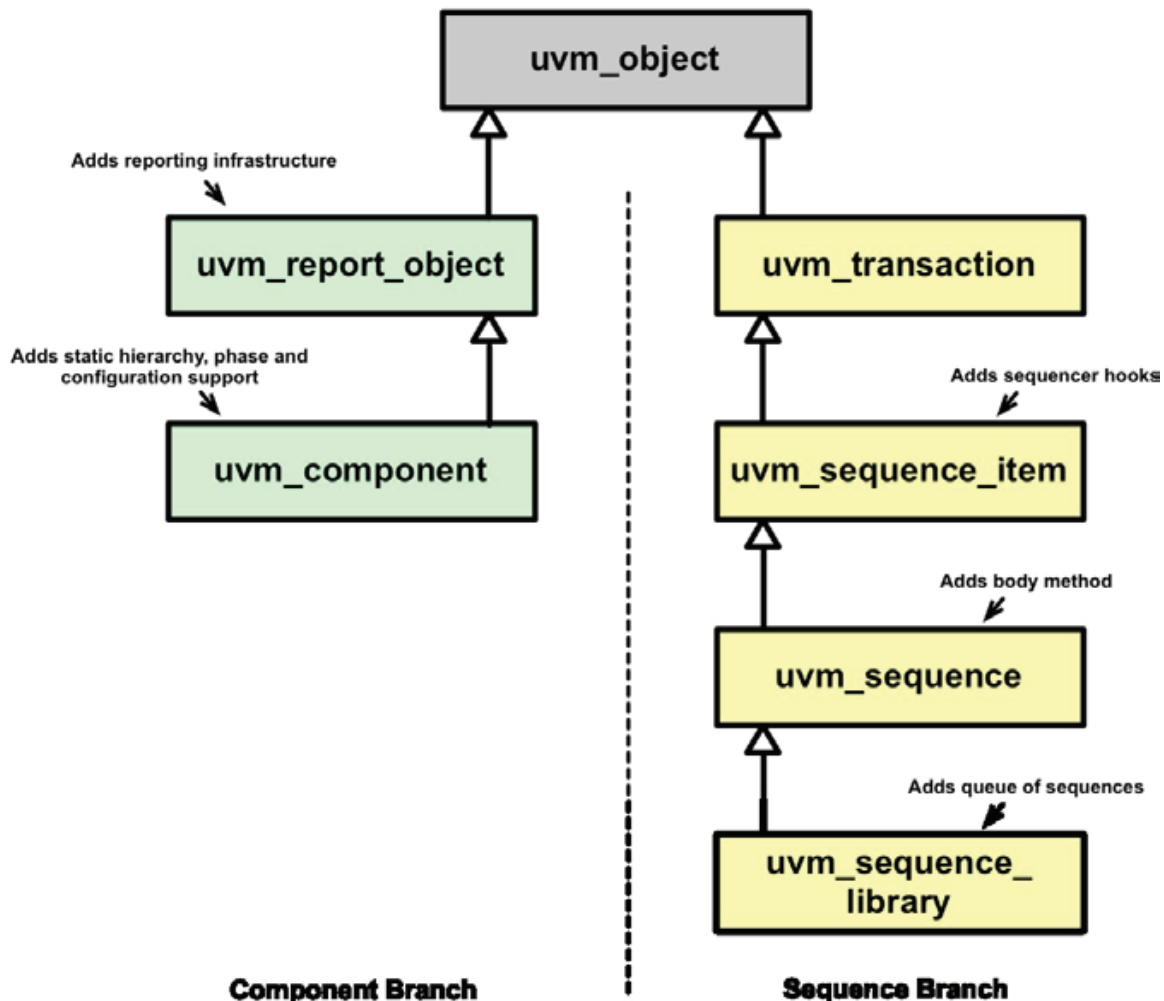
endclass : test1_seq

```

The Sequence Library

UVM provides a class for randomly creating and running sequences. This class is called `uvm_sequence_library`. The `uvm_sequence_library` class inherits from `uvm_sequence` which means that an instance of a sequence library is also a sequence. This is important because after a sequence library is configured, it is used just as a sequence is used.

Simplified UVM Inheritance Diagram



Whilst the sequence library is available to use, it is not the preferred method for creating and starting sequences and it is not recommended. The techniques described in the Sequences/Generation article generally should be used as those techniques provide easy control over serial and parallel execution of sequences. The techniques described on the Sequences/Generation page also allows for the maximum reuse of a test stimulus.

The `uvm_sequence_library` class is replacing the sequencer library functionality which is deprecated in UVM. The sequencer library was used by some in OVM, but it had several limitations.

Functionality

The `uvm_sequence_library` class provides a means for randomly selecting a sequence to run from a list of sequences registered with it. The total number of sequences run when the sequence library is started is controllable with a default of 10.

Basics

To create a sequence library, a class is extended from the parameterized `uvm_sequence_library` class. There are two parameter values which can potentially be set. They are the same REQ and RSP parameter values that a sequence requires. Also like a sequence, the RSP parameter defaults to having the same value as the REQ parameter.

After extending from the `uvm_sequence_library` class, then the standard factory registration takes place using the ``uvm_object_utils()` macro. Then a unique ``uvm_sequence_library_utils()` macro needs to be invoked. The macro and function are needed to populate the sequence library with any sequences that were statically registered with it or any of its base classes.

```
class mem_seq_lib extends uvm_sequence_library #(mem_item);
  `uvm_object_utils(mem_seq_lib)
  `uvm_sequence_library_utils(mem_seq_lib)

  function new(string name="mem_seq_lib");
    super.new(name);
  endfunction : new

endclass : mem_seq_lib
```

Registering Sequences

Sequences can be registered with a sequence library either by using a macro or function calls. Mentor recommends using function calls as they are more flexible and do not require extra code inside of standard sequences. The macro method requires adding a macro call (``uvm_add_to_seq_lib(<sequence_type>, <seq_library_type>)`) to standard sequences which may already exist or which may have been written by someone else.

To register a sequence(s) with every instance of a particular type of sequence library, the `add_typewise_sequence()` and/or `add_typewise_sequences()` functions are used.

```
class mem_seq_lib extends uvm_sequence_library #(mem_item);
  ...

  function new(string name="mem_seq_lib");
    super.new(name);

    //Explicitly add the memory sequences to the library
    add_typewise_sequences({mem_seq1::get_type(),
mem_seq2::get_type(), mem_seq3::get_type(),
                           mem_seq4::get_type(),
mem_seq5::get_type(), mem_seq6::get_type()});

  endfunction : new
```

```
endclass : mem_seq_lib
```

The most convenient location to place the `add_typewide_sequence()` and/or `add_typewide_sequences()` call is in the constructor of the `sequence_library`.

Sequences can also be registered with individual instances of a sequence library by using the `add_sequence()` or the `add_sequences()` function. This typically would be done in the test where the sequence is instantiated.

Sequences can be removed from the library using the `remove_sequence()` API.

```
class test_seq_lib extends test_base;

...

task main_phase(uvm_phase phase);
    phase.raise_objection(this, "Raising Main Objection");

    //Register another sequence with this sequence library instance
    seq_lib.add_sequence( mem_error_seq::get_type() );

    //Start the mem sequence
    seq_lib.start(m_mem_sequencer); //This task call is blocking

    phase.drop_objection(this, "Dropping Main Objection");
endtask : main_phase

endclass : test_seq_lib
```

Controls

The `uvm_sequence_library` class provides some built-in controls to constrain the sequence library when it is `randomized()`. To control the number of sequences which are run when the sequence library is started, the data members `min_random_count` (default of 10) and `max_random_count` (default of 10) can be changed. These values can either be changed in the test where the sequence is instantiated

```
class test_seq_lib extends test_base;

...

task main_phase(uvm_phase phase);
    phase.raise_objection(this, "Raising Main Objection");

    //Configure the constraints for how many sequences are run
    seq_lib.set_min_random_count(5);
    seq_lib.set_max_random_count(12);

    //Randomize the sequence library
    if (!seq_lib.randomize())
        `uvm_error(report_id, "The mem_seq_lib library failed to
randomize()")
```

```

    //Start the mem sequence
    seq_lib.start(m_mem_sequencer); //This task call is blocking

    phase.drop_objection(this, "Dropping Main Objection");
endtask : main_phase

endclass : test_seq_lib

```

or by using the `uvm_config_db` and specifying the full path to the sequence library. This second option is only used if the sequence library is configured to be a default sequence for an UVM phase which is not recommended by Mentor.

The method of selection for the next sequence to be executed when the sequence library is running can also be controlled. Four options exist and are specified in an enumeration.

Enum Value	Description
UVM_SEQ_LIB_RAND	Random sequence selection. This is the default.
UVM_SEQ_LIB_RANDC	Random cyclic sequence selection
UVM_SEQ_LIB_ITEM	Emit only items, no sequence execution
UVM_SEQ_LIB_USER	Apply a user-defined random-selection algorithm

To change the randomization method used, the `selection_mode` data member is set before the sequence library is started.

```

class test_seq_lib extends test_base;

    ...

    task main_phase(uvm_phase phase);
        phase.raise_objection(this, "Raising Main Objection");

        //Change to RANDC mode for selection
        seq_lib.set_selection_mode(UVM_SEQ_LIB_RANDC);

        //Start the mem sequence
        seq_lib.start(m_mem_sequencer); //This task call is blocking

        phase.drop_objection(this, "Dropping Main Objection");
    endtask : main_phase

endclass : test_seq_lib

```

Sequence-Driver Use Models

Unidirectional Non-Pipelined

In the unidirectional non-pipelined use model, requests are sent to the driver, but no responses are received back from the driver. The driver itself may use some kind of handshake mechanism as part of its transfer protocol, but the data payload of the transaction is unidirectional.

An example of this type of use model would be an unidirectional communication link such as an ADPCM or a PCM interface.

Bidirectional Non-Pipelined

In the bidirectional non-pipelined use model, the data transfer is bidirectional with a request sent from a sequence to a driver resulting in a response being returned to the sequence from the driver. The response occurs in lock-step with the request and only one transfer is active at a time.

An example of this type of use model is a simple bus interface such as the AMBA Peripheral Bus (APB).

Pipelined

In the pipelined use model, the data transfer is bidirectional, but the request phase overlaps the response phase to the previous request. Using pipelining can provide hardware performance advantages, but it complicates the sequence driver use model because requests and responses need to be handled separately.

CPU buses frequently use pipelines and one common example is the AMBA AHB bus.

Out Of Order Pipelined

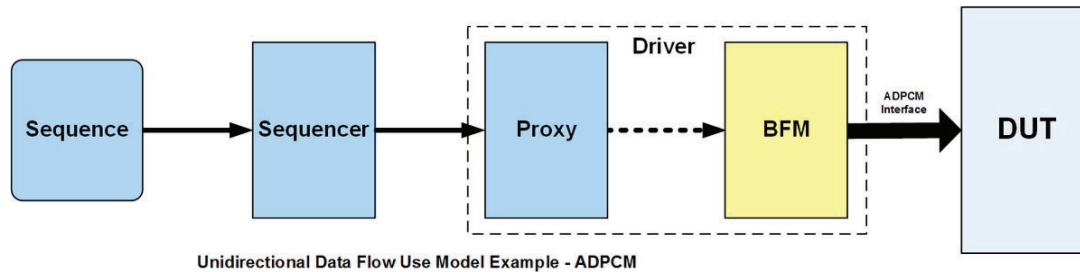
In applications such as fabrics and switches, requests are acknowledged but not responded to in order. The reason being that an accessed resource may already be busy or the path to it may already be in use, and the requesting master has its command accepted leaving it free to issue a new request. This requires a more complicated use model where queues and ids are used to track outstanding responses.

Another variation of this use model would be interleaved bursts where a burst transfer can be interleaved with other transfers.

Advanced SoC buses such as AXI and OCP support out of order responses.

Unidirectional Protocols

For a driver, composed of a BFM-proxy pair in the dual domain testbench, the driver proxy controls the flow of sequence_items by using `get_next_item()` to obtain the next sequence_item to be processed, and making the `item_done()` call only once it has finished processing the item. The sequence is blocked at its `finish_item()` call until the `item_done()` call is made by the driver proxy.



In the unidirectional non-pipelined sequence-driver use model, it is the data flow that is unidirectional. The sequence provides a series of request sequence_items to be driven on the DUT interface, but receives no response sequence_items. However, the control flow of this use model is bidirectional, since there are handshake mechanisms built into the UVM sequencer communication protocol. The driver BFM may also implement a handshake on the DUT interface, but this will not be visible to the controlling sequence.

A Unidirectional Example

An example of a unidirectional dataflow is sending ADPCM packets using a PCM framing protocol. The waveform illustrates the protocol.

```
class adpcm_driver extends uvm_driver #(adpcm_seq_item);

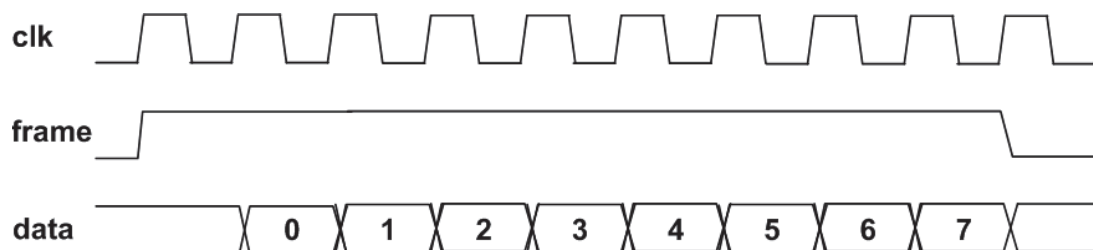
`uvm_component_utils(adpcm_driver)

adpcm_seq_item req;

local virtual adpcm_driver_bfm m_bfm;

function new(string name = "adpcm_driver", uvm_component parent =
null);
    super.new(name, parent);
endfunction

function void set_bfm(virtual adpcm_driver_bfm bfm);
    m_bfm = bfm;
```



ADPCM Frame Timing Diagram

```

endfunction: set_bfm

task run_phase(uvm_phase phase);
    forever begin
        seq_item_port.get_next_item(req); // Gets the sequence_item from
        the sequence
        m_bfm.drive(req);
        seq_item_port.item_done(); // Indicates that the sequence_item has
        been consumed
    end
endtask: run_phase

endclass: adpcm_driver

```

```

interface adpcm_driver_bfm (adpcm_if ADPCM);

import adpcm_pkg::*;

initial begin
    ADPCM.frame <= 0;
    ADPCM.data <= 0;
end

task drive(adpcm_seq_item req);
    repeat (req.delay) begin // Delay between packets
        @(posedge ADPCM.clk);
    end

    ADPCM.frame <= 1; // Start of frame

    for (int i = 0; i < 8; i++) begin // Send nibbles
        @(posedge ADPCM.clk); ADPCM.data
        <= req.data[3:0]; req.data = req.data >> 4;
    end

    ADPCM.frame <= 0; // End of frame
endtask: drive

endinterface: adpcm_driver_bfm

```

The sequence implementation in this case is a loop which generates a series of sequence_items. A variation on this theme would be for the sequence to actively shape the traffic sent rather than send purely random stimulus.

```

class adpcm_tx_seq extends uvm_sequence #(adpcm_seq_item);

    `uvm_object_utils (adpcm_tx_seq)

    // ADPCM sequence_item

```

```
adpcm_seq_item req;

// Controls the number of request sequence items sent
rand int no_reqs = 10;

function new(string name = "adpcm_tx_seq");
    super.new(name);
endfunction

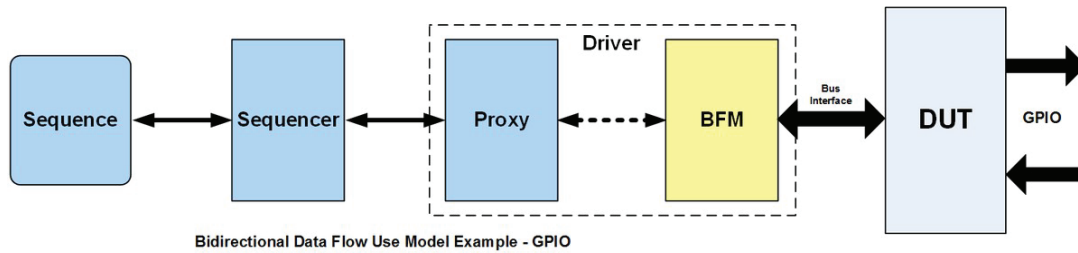
task body;
    req = adpcm_seq_item::type_id::create("req");

    for (int i = 0; i < no_reqs; i++) begin
        start_item(req);
        if (!req.randomize()) begin
            `uvm_fatal("body", "req randomization failure")
        end
        finish_item(req);
        `uvm_info("ADPCM_TX_SEQ_BODY", $sformatf("Transmitted frame %0d",
i), UVM_LOW)
    end
endtask: body

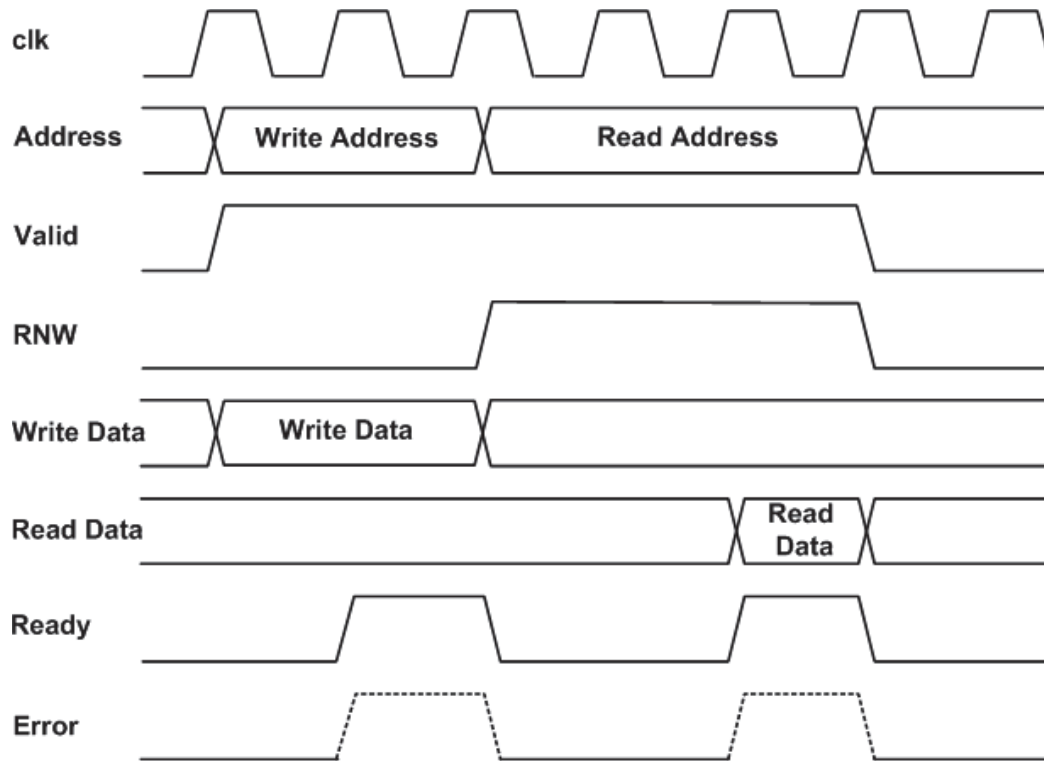
endclass: adpcm_tx_seq
```

Bidirectional Protocols

For a driver, composed of a BFM-proxy pair in the dual domain testbench, one of the most common sequence-driver use cases is where the sequencer sends request sequence_items to the driver proxy, which then executes the request phase of the pin-level protocol through the driver BFM. The driver BFM in turn captures the response phase of the pin-level transaction, and returns the response back to the sequence via the driver proxy. In this scenario the flow of data is bidirectional, and a new request phase cannot be started until the response phase has completed. An example of this kind of protocol would be a simple peripheral bus such as the AMBA APB.



To illustrate how this use model would be implemented, a DUT containing a GPIO and a bus interface is used. The bus protocol is shown in the timing diagram. The request phase of the transaction is initiated by asserting the valid signal, with the address and direction signal (RNW) indicating the type of bus transfer to take place. The response phase of the transaction is completed when the ready signal becomes active.



Coupled Request - Response Bus Protocol

The driver proxy that manages this protocol will collect a request sequence_item from the sequencer and then drive the bus request phase. The driver BFM waits until the interface ready line becomes active before capturing the response information, which consists here of the error status bit and the read data if a read has just taken place. The recommended way of implementing the driver proxy is to use `get_next_item()` followed by `item_done()` as per the following example:


```

class bidirect_bus_driver extends uvm_driver #(bus_seq_item);

`uvm_component_utils(bidirect_bus_driver) bus_seq_item

req;

local virtual bidirect_bus_driver_bfm m_bfm;

function new(string name = "bidirect_bus_driver", uvm_component parent
= null);
    super.new(name, parent);
endfunction

function void set_bfm(virtual bidirect_bus_driver_bfm bfm); m_bfm = bfm;
endfunction: set_bfm

task run_phase(uvm_phase phase);
    m_bfm.wait_for_reset();

    forever begin
        seq_item_port.get_next_item(req); // Start processing req item
        m_bfm.drive(req);
        seq_item_port.item_done(); // End of req item
    end
endtask: run_phase

endclass: bidirect_bus_driver

interface bidirect_bus_driver_bfm (
    input    logic        clk,
    input    logic        resetn, output
    logic [31:0] addr, output logic [31:0]
    write_data, output logic    rnw,
    output logic            valid,
    input    logic        ready, input
    logic [31:0] read_data, input
    logic            error
);

import bidirect_bus_pkg::*; initial

begin
    BUS.valid <= 0;
    BUS.rnw <= 1;
end

```

```

task wait_for_reset();
    @(posedge BUS.resetn);
endtask: wait_for_reset

task drive(bus_seq_item req);
    repeat (req.delay) begin // Delay between bus transactions
        @(posedge BUS.clk);
    end

    BUS.valid <= 1;

    BUS.addr <= req.addr;
    BUS.rnw <= req.read_not_write;
    if (req.read_not_write == 0) begin
        BUS.write_data <= req.write_data;
    end

    while (BUS.ready != 1) begin
        @(posedge BUS.clk);
    end

    // At end of the pin level bus transaction
    // Copy response data into the req fields:
    if (req.read_not_write == 1) begin
        req.read_data = BUS.read_data; // Copy read data response
    end
    req.error = BUS.error; // Copy bus error response status

    BUS.valid <= 0; // End the pin level bus transaction
endtask: drive

endinterface: bidirect_bus_driver_bfm

```

Note that the driver proxy is sending back the response to the sequence implicitly through the pertinent response fields within the req sequence_item, by way of the driver BFM updating these fields as part of the drive() call, before the item_done() call. At the sequence end of the transaction, the sequence is blocked in the finish_item() call until the item_done() call occurs. Once the sequence is unblocked, its req handle is still pointing to the req object which has had its response fields updated by the driver BFM. This means that the sequence can reference the response fields of the req sequence_item.

```

class bus_seq extends uvm_sequence #(bus_seq_item);

    `uvm_object_utils(bus_seq)

    bus_seq_item req;

    rand int limit = 40; // Controls the number of iterations

```

```

function new(string name = "bus_seq");
    super.new(name);
endfunction

task body();
    req = bus_seq_item::type_id::create("req");

    repeat (limit) begin
        start_item(req);
        // The address is constrained to be within the address of the GPIO
function
    // within the DUT, The result will be a request item for a read or
a write
        assert(req.randomize() with {addr inside
{[32'h0100_0000:32'h0100_001C]}});
        finish_item(req);
        // The req handle points to the object that the driver has updated
with response data
        `uvm_info("seq_body", req.convert2string(), UVM_MEDIUM);
    end
endtask: body

endclass: bus_seq

```

Alternative Implementation Option

Although an alternative implementation is discussed below, the recommended way to implement this sequence driver use model is as in the preceding code.

Driver put, Sequence get_response

The code in the driver proxy could use a get() method call to collect the request sequence_item, which would unblock the finish_item() call in the sequence execution. However, the driver proxy must then use the put() method to signal back to the sequence that the bus transfer has completed, and the sequence must use the blocking get_response() method call to wait for the driver proxy to notify completion of the transfer. Any response information from the pin-level bus transaction can be sent from the driver proxy to the sequence via the argument to the put() method.

```

// Alternative version of the driver proxy run method
task run_phase(uvm_phase phase);
    m_bfm.wait_for_reset();

    forever begin
        seq_item_port.get(req); // Start processing req item
        m_bfm.drive(req, rsp);
        rsp.set_id_info(req); // Set the rsp id = req id
        seq_item_port.put(rsp); // put returns the response
    end
endtask: run_phase

```

// Corresponding alternative version of the driver BFM drive task

```
task drive(bus_seq_item req, output bus_seq_item rsp);
    repeat (req.delay) begin // Delay between bus transactions
        @(posedge BUS.clk);
    end

    BUS.valid <= 1; BUS.addr <=

    req.addr;
    BUS.rnw <= req.read_not_write;
    if (req.read_not_write == 0) begin
        BUS.write_data <= req.write_data;
    end

    while (BUS.ready != 1) begin
        @(posedge BUS.clk);
    end

    // At end of the pin level bus transaction
    // Copy request with response data into the rsp
    $cast(rsp, req.clone()); // Clone the req
    if (req.read_not_write == 1) begin
        rsp.read_data = BUS.read_data; // Copy read data response
    end
    rsp.error = BUS.error; // Copy bus error response status

    BUS.valid <= 0; // End the pin level bus transaction
endtask: drive
```

// Corresponding version of the sequence body method

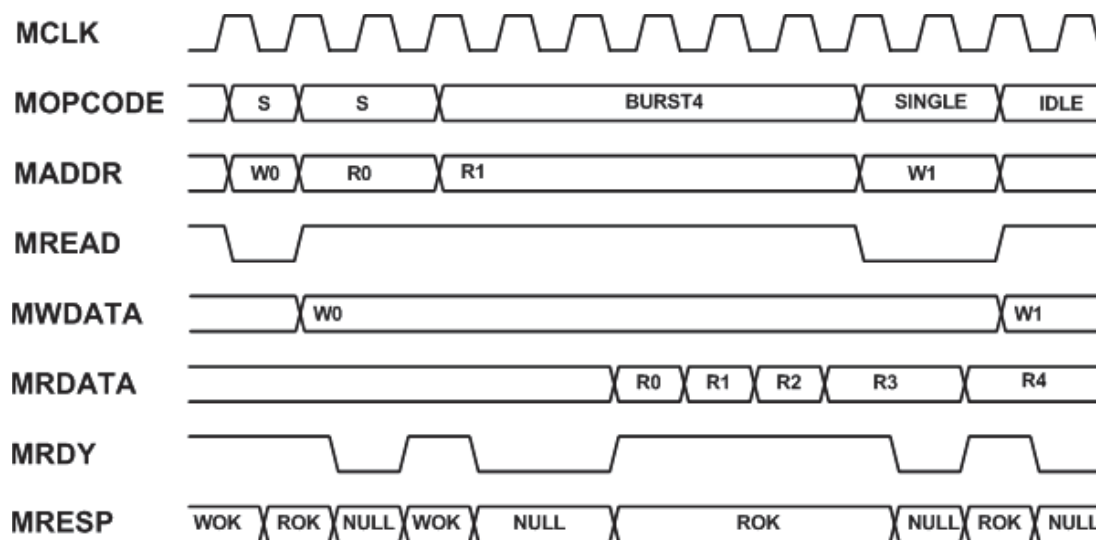
```
task body;
    req = bus_seq_item::type_id::create("req");

    repeat (limit) begin
        start_item(req);
        // The address is constrained to be within the address of the GPIO
    function
        // within the DUT, The result will be a request item for a read or
        a write
        assert(req.randomize() with {addr inside
            {[32'h0100_0000:32'h0100_001C]}});
        finish_item(req);
        get_response(rsp);
        // The rsp handle points to the object that the driver has updated
    with response data
        `uvm_info("seq_body", rsp.convert2string(), UVM_MEDIUM);
    end endtask:
body
```

For more information on this implementation style, especially how to initialize the response item, see the section on the get-put use model in the Driver/Sequence API article.

Pipelined Protocols

In a pipelined bus protocol a data transfer is broken down into two or more phases which are executed one after the other, often using different groups of signals on the bus. This type of protocol allows several bus transfers to be in progress at the same time, with each transfer occupying one stage of the pipeline. The AMBA AHB bus is an example of a pipelined bus with two phases: the address phase and the data phase. During the address phase, the address and the bus control information, such as the opcode, is set up by the host, and then during the data phase the data transfer between the target and the host takes place. While the data phase for one transfer is taking place on the second stage of the pipeline, the address phase for the next transfer can take place on the first stage of the pipeline. Other protocols such as OCP use more phases.



Pipelined bus protocol – Timing diagram

A pipelined protocol has the potential to increase the bandwidth of a system, since it increases the number of transfers that can take place over a given number of clock cycles, provided the pipeline is kept full. Using a pipeline also relaxes the timing requirements for target devices since it gives them extra time to decode and respond to a host access.

A pipelined protocol driver, composed of a BFM-proxy pair in the dual domain testbench, could be modelled with a simple bidirectional style whereby the sequence sends a sequence item to the driver proxy which unblocks the sequence when the corresponding bus transaction has completed. In reality, most I/O and register style accesses take place in this way. The drawback is that it lowers the bandwidth of the bus and does not stress-test it. In order to implement a pipelined sequence-driver use model, a number of design considerations need to be taken into account in order to support fully pipelined transfers:

- **Driver Implementation** - The driver (BFM or proxy) needs to have multiple threads running, where each thread takes a sequence item through one of the pipeline stages.
- **Keeping the pipeline full** - The driver needs to unblock the sequencer in a timely manner to get the next sequence item so that the pipeline can be kept full.
- **Sequence Implementation** - The sequence needs to have separate stimulus request and response threads, with the former continually generating new bus transactions for the driver to keep the pipeline full.

Recommended Implementation Pattern Using Get and Put

A simple way to model a pipelined protocol with a sequence and driver is to use the `get()` and `put()` methods from the driver-sequencer API.

Driver Implementation

In order to support pipelining, a driver needs to be able to process multiple `sequence_items` concurrently. A simple example to achieve this is by a driver proxy continually taking a `sequence_item` and starting the associated bus transfer - pipeline stage 1 - then promptly reiterating for the next item *before* full completion of the initiated transfer - pipeline stage 2. Each stage in the pipeline requires a separate thread, in this case one thread to `get()` a next bus transaction and execute its address/command portion, and the second thread to complete its data portion (in parallel with the address/command portion of the subsequent bus transaction).

To ensure that no more than one bus transaction enters each pipeline stage at a time, several complementary techniques exist to implement mutual exclusion. For instance, exclusive access to the address/command phase of a bus transaction can be accomplished through a blocking call from the driver proxy (`task begin_transfer(...)`) upon getting the next `sequence_item` to execute this first phase. Subsequent exclusive access to the concurrently threaded data phase of the bus transaction triggered by completion of the address/command phase can then be guaranteed using a simple semaphore-based locking mechanism. The semaphore is grabbed when available prior to executing the data phase, and released again upon phase completion. Note that at the end of this final phase of the bus transaction, a response must be "put()" back to the originating sequence to satisfy the UVM sequencer-driver handshake protocol (`function end_transfer(...)`).

The following code example shows the two-stage pipeline described above to further illustrate these principles.

```
class mbus_pipelined_driver extends uvm_driver #(mbus_seq_item);

`uvm_component_utils(mbus_pipelined_driver)

local virtual mbus_pipelined_driver_bfm m_bfm;

local mbus_seq_item pipeline [$];

function new(string name = "mbus_pipelined_driver", uvm_component
parent = null);
    super.new(name, parent);
endfunction

function void set_bfm(virtual mbus_pipelined_driver_bfm bfm);
    m_bfm = bfm;
    m_bfm.proxy = this;
endfunction: set_bfm

task run_phase(uvm_phase phase);
    m_bfm.wait_for_reset();

    do_pipelined_transfers();
endtask

task do_pipelined_transfers();
```

```

mbus_seq_item req;

forever begin
    seq_item_port.get(req);
    accept_tr(req, $time);
    void!(begin_tr(req, "pipelined_driver"));

    // This blocking call performs the command phase of the request and
    then returns
    // right away before completing the data phase, thus allowing the
command phase
    // of the subsequent request (next loop iteration) to occur in
parallel with the
    // data phase of the current request, and so implementing the
pipeline
    m_bfm.begin_transfer(req);

    pipeline.push_back(req);
end
endtask: do_pipelined_transfers

// Function to complete the sequence item - driver handshake back to
the sequence,
// decoupled from the point of the originating request
function void end_transfer(mbus_seq_item req); mbus_seq_item rsp
    = pipeline.pop_front(); rsp.copy(req);

    seq_item_port.put(rsp); // End of req item // put_response(rsp); is
actually a function instead of task
    end_tr(rsp);
endfunction: end_transfer

endclass: mbus_pipelined_driver

interface mbus_pipelined_driver_bfm (mbus_if MBUS); import

mbus_pipelined_agent_pkg::*; mbus_pipelined_driver proxy;

mbus_seq_item current_tr;

event do_data_phase;

task begin_transfer(mbus_seq_item req);
    command_phase(req);

```

```

    pipeline_lock_get(); // Start of data phase: grab semaphore

    current_tr = req;
    ->do_data_phase;
endtask: begin_transfer

always begin
    @do_data_phase;

    @(posedge MBUS.MCLK);

    data_phase(current_tr);
    proxy.end_transfer(current_tr);

    pipeline_lock_put(); // End of data phase: release semaphore
end

task wait_for_reset();
    ...
endtask

task command_phase(mbus_seq_item req);
    MBUS.MADDR <= req.MADDR;
    MBUS.MREAD <= req.MREAD;
    MBUS.MOPCODE <= req.MOPCODE;
    @(posedge MBUS.MCLK);
    ...
endtask: command_phase

task data_phase(mbus_seq_item req);
    ...
    if (req.MREAD == 1) begin
        req.MRDATA = MBUS.MRDATA;
    end
endtask: data_phase

bit pipeline_lock = 0;

task pipeline_lock_get();
    while (pipeline_lock) @(posedge MBUS.MCLK);
    pipeline_lock = 1;
endtask: pipeline_lock_get

function void pipeline_lock_put(); pipeline_lock
    = 0;
endfunction: pipeline_lock_put

```



```
endinterface: mbus_pipelined_driver_bfm
```

Sequence Implementation

Non-pipelined Accesses

Often non-pipelined transfers are required because a typical bus fabric is mimicking what a software program does, namely accessing individual locations. For instance using the value read back from one location to determine what to do next in terms of reading or writing other locations. For a non-pipelined UVM sequence that would work with the pipelined driver, the sequence body() method would call start_item(), finish_item() and get_response() methods in order, as illustrated in the code example below. The get_response() method blocks until the driver proxy provides a response using its put() method at the end of the bus transaction.

```
// This sequence shows how a series of non-pipelined accesses to the
bus would work.
// The sequence waits for each item to finish before starting the next.
class mbus_unpipelined_seq extends uvm_sequence #(mbus_seq_item);

`uvm_object_utils(mbus_nonpipelined_seq)

logic[31:0] addr[10]; // To save addresses
logic[31:0] data[10]; // To save data for checking

int error_count;

function new(string name = "mbus_nonpipelined_seq");
    super.new(name);
endfunction

task body;
    mbus_seq_item req = mbus_seq_item::type_id::create("req");
    error_count = 0;

    for (int i=0; i<10; i++) begin
        start_item(req);
        assert(req.randomize() with {MREAD == 0; MOPCODE == SINGLE; MADDR
inside {[32'h0010_0000:32'h001F_FFFC]};});
        addr[i] = req.MADDR;
        data[i] = req.MWDATA;
        finish_item(req);
        get_response(req);
        `uvm_info("", $sformatf("write (i = %0d) of %h at %h", i,
req.MWDATA, req.MADDR), UVM_MEDIUM);
    end

    foreach (addr[i]) begin
        start_item(req);
        req.MADDR = addr[i];
        req.MREAD = 1;
```

```

    finish_item(req);
    get_response(req);
    `uvm_info("", $sformatf("read (i = %0d) of %h at %h", i,
req.MRDATA, req.MADDR), UVM_MEDIUM);
    if (data[i] != req.MRDATA) begin
        error_count++;
        `uvm_error("body", $sformatf("@%0h Expected data:%0h Actual
data:%0h", addr[i], data[i], req.MRDATA))
    end
end
endtask: body

endclass: mbus_nonpipelined_seq

```

Note: This example sequence has data checking built-in, merely to demonstrate how the read data value can be used. Normally, the specific type of checking required would be done using a separate scoreboard.

Pipelined Accesses

Pipelined accesses are primarily used to stress-test the bus and require a different approach in the sequence. A pipelined sequence needs to have separate threads for generating the request sequence_items and handling the response sequence_items.

The generation loop will block on each finish_item() call until one of the threads in the driver proxy completes a get() call. Once the loop is unblocked it needs to generate a new sequence_item to provide something for the next driver proxy thread to get(). Note that a new request sequence_item needs to be generated in each iteration of the loop; if only one request item handle were used, the driver proxy would be attempting to execute the request while the sequence is modifying its fields.

There is no response handling in the given example sequence. The assumption here is that data-checking will be done by a separate scoreboard as noted above. However, with the get() and put() driver implementation, there is a response FIFO in the sequence that must be managed. In the example, the response_handler is enabled using the use_response_handler() method, and the response_handler function is then called each time a response becomes available, keeping the sequence response FIFO empty. In this case the response_handler keeps count of the number of transactions to ensure that the sequence only exits upon completion of the final transaction.

```

// This is a pipelined version of the previous sequence with no
blocking call to get_response().
// There is no intent to check the data, which would be carried out by
a scoreboard.
class mbus_pipelined_seq extends uvm_sequence #(mbus_seq_item);

    `uvm_object_utils(mbus_pipelined_seq)

    logic[31:0] addr[10]; // To save addresses
    int count;           // To ensure that the sequence does not complete
    prematurely

    function new(string name = "mbus_pipelined_seq");
        super.new(name);
    endfunction

```

```

task body;
    mbus_seq_item req = mbus_seq_item::type_id::create("req");
    use_response_handler(1);
    count = 0;

    for (int i=0; i<10; i++) begin
        start_item(req);
        assert(req.randomize() with {MREAD == 0; MOPCODE == SINGLE; MADDR
inside {[32'h0010_0000:32'h001F_FFFC]};});
        addr[i] = req.MADDR;
        finish_item(req);
        `uvm_info("", $sformatf("write (i = %0d) of %h at %h", i,
req.MWDATA, req.MADDR), UVM_MEDIUM);
    end

    foreach (addr[i]) begin
        start_item(req);
        req.MADDR = addr[i];
        req.MREAD = 1;
        finish_item(req);
        `uvm_info("", $sformatf("read (i = %0d) of %h at %h", i,
req.MRDATA, req.MADDR), UVM_MEDIUM);
    end

    wait(count == 20); // Do not end the sequence until the last req item
    is complete
endtask: body

// The response_handler function serves to keep the sequence response
FIFO empty
function void response_handler(uvm_sequence_item response);
    count++;
endfunction: response_handler

endclass: mbus_pipelined_seq
    
```

If the sequence needs to handle responses, the response_handler function should be extended.

Alternative Implementation Pattern Using Events To Signal Completion

Adding Completion Events to sequence_items

In this implementation pattern, events are added to the sequence_item to provide a means for the driver proxy to signal to the sequence that it has completed a specific transaction phase. In the example below, a uvm_event_pool is used for the events along with two methods to trigger and to wait for events in the pool.

```
// The mbus_seq_item is designed to be used with a pipelined bus
driver. It contains an event pool
// which is used to signal back to the sequence when the driver has
completed different pipeline stages
```

```
class mbus_seq_item extends uvm_sequence_item;
```

```
// From the master to the slave
```

```
rand logic[31:0] MADDR; rand
logic[31:0] MWDATA; rand logic
MREAD;
rand mbus_opcode_e MOPCODE;
```

```
// Driven by the slave to the master
```

```
mbus_resp_e MRESP;
logic[31:0] MRDATA;
```

```
// Event pool
```

```
uvm_event_pool events;
```

```
`uvm_object_utils(mbus_seq_item)
```

```
function new(string name = "mbus_seq_item");
    super.new(name);
    events = get_event_pool();
endfunction
```

```
constraint addr_is_32 {MADDR[1:0] == 0;}
```

```
// Wait for an event - called by sequence
```

```
task wait_trigger(string evnt); uvm_event e =
    events.get(evnt); e.wait_trigger();
endtask: wait_trigger
```

```
// Trigger an event - called by driver
```

```
function void trigger(string evnt); uvm_event e =
    events.get(evnt); e.trigger();
endfunction: trigger
```

```
// do_copy(), do_compare etc
```

```
endclass: mbus_seq_item
```

Driver Signalling Completion Using sequence_item Events

The driver proxy is almost identical to the get-put implementation above. Instead of using the put() method, it triggers the phase-done notification events in the sequence_item to signal phase completion back to the sequence, and make response information available via the sequence_item handle.

```
class mbus_pipelined_driver extends uvm_driver #(mbus_seq_item);

...

task do_pipelined_transfers();
    mbus_seq_item req;

    forever begin
        seq_item_port.get(req);
        accept_tr(req, $time);
        void'(begin_tr(req, "pipelined_driver"));

        // This blocking call performs the command phase of the request and
        then returns
        // right away before completing the data phase, thus allowing the
command phase
        // of the subsequent request (next loop iteration) to occur in
parallel with the
        // data phase of the current request, and so implementing the
pipeline
        m_bfm.begin_transfer(req);

        pipeline.push_back(req);

        req.trigger("CMD_DONE"); // Signal CMD_DONE at this end of command
phase
    end
endtask: do_pipelined_transfers

// Function to complete the sequence item - driver handshake back to
the sequence,
// decoupled from the point of the originating request
function void end_transfer(mbus_seq_item req); mbus_seq_item rsp
    = pipeline.pop_front(); rsp.copy(req);

    rsp.trigger("DATA_DONE"); // Signal DATA_DONE at this end of data
phase
    end_tr(rsp);
endfunction: end_transfer
```

```
endclass: mbus_pipelined_driver
```

Non-pipelined Access Sequences

Non-pipelined accesses are made from sequences which block, after completing the `finish_item()` call, by waiting for the event that notifies data phase completion. This enables code in the sequence `body()` method to react to the data read back. An alternative way of implementing this type of sequence would be to overload the `finish_item()` method so that it does not return until the data phase completion event occurs.

```
// Task: finish_item
// Calls super.finish_item() but then also waits for the sequence
item's data phase event.
// This is notified by the driver when it has completely finished
processing the item.
task finish_item( uvm_sequence_item item , int set_priority = -1 );
    super.finish_item( item , set_priority );    // The "normal"
finish_item()

    item.wait_trigger("DATA_DONE");    // Wait for the data phase to
complete
endtask: finish_item
```

As in the previous example of an non-pipelined sequence, the code example below has a data integrity check purely for illustrative purposes.

```
class mbus_unpipelined_seq extends uvm_sequence #(mbus_seq_item);

`uvm_object_utils(mbus_unpipelined_seq)

logic[31:0] addr[10]; // To save addresses
logic[31:0] data[10]; // To save data

int error_count;

function new(string name = "mbus_unpipelined_seq");
    super.new(name);
endfunction

task body;
    mbus_seq_item req = mbus_seq_item::type_id::create("req");
    error_count = 0;

    for (int i=0; i<10; i++) begin
        start_item(req);
        assert(req.randomize() with {MREAD == 0; MOPCODE == SINGLE; MADDR
inside { [32'h0010_0000:32'h001F_FFFC] }; });
        addr[i] = req.MADDR;
        data[i] = req.MWDATA;
        finish_item(req);
    end
endtask
```

```

    req.wait_trigger("DATA_DONE");
    `uvm_info("", $sformatf("write (i = %0d) of %h at %h", i,
req.MWDATA, req.MADDR), UVM_MEDIUM);
end

foreach (addr[i]) begin
    start_item(req);
    req.MADDR = addr[i];
    req.MREAD = 1;
    finish_item(req);
    req.wait_trigger("DATA_DONE");
    `uvm_info("", $sformatf("read (i = %0d) of %h at %h", i,
req.MRDATA, req.MADDR), UVM_MEDIUM);
    if (req.MRDATA != data[i]) begin
        error_count++;
        `uvm_error("body", $sformatf("@%0h Expected data:%0h Actual
data:%0h", addr[i], data[i], req.MRDATA))
    end
end
endtask: body

endclass: mbus_unpipelined_seq

```

Pipelined Access

The pipelined access sequence does not wait for the data phase completion event notification to generate the next sequence item. Unlike for the get-put driver model, there is no need to manage the response FIFO, and so this implementation is a little more straightforward in this respect.

```

class mbus_pipelined_seq extends uvm_sequence #(mbus_seq_item);

    `uvm_object_utils(mbus_pipelined_seq)

    logic[31:0] addr[10]; // To save addresses

    function new(string name = "mbus_pipelined_seq");
        super.new(name);
    endfunction

    task body;
        mbus_seq_item req = mbus_seq_item::type_id::create("req");

        for (int i=0; i<10; i++) begin
            start_item(req);
            assert(req.randomize() with {MREAD == 0; MOPCODE == SINGLE; MADDR
inside {[32'h0010_0000:32'h001F_FFFC]}});
            addr[i] = req.MADDR;
            finish_item(req);
            `uvm_info("", $sformatf("write (i = %0d) of %h at %h", i,

```

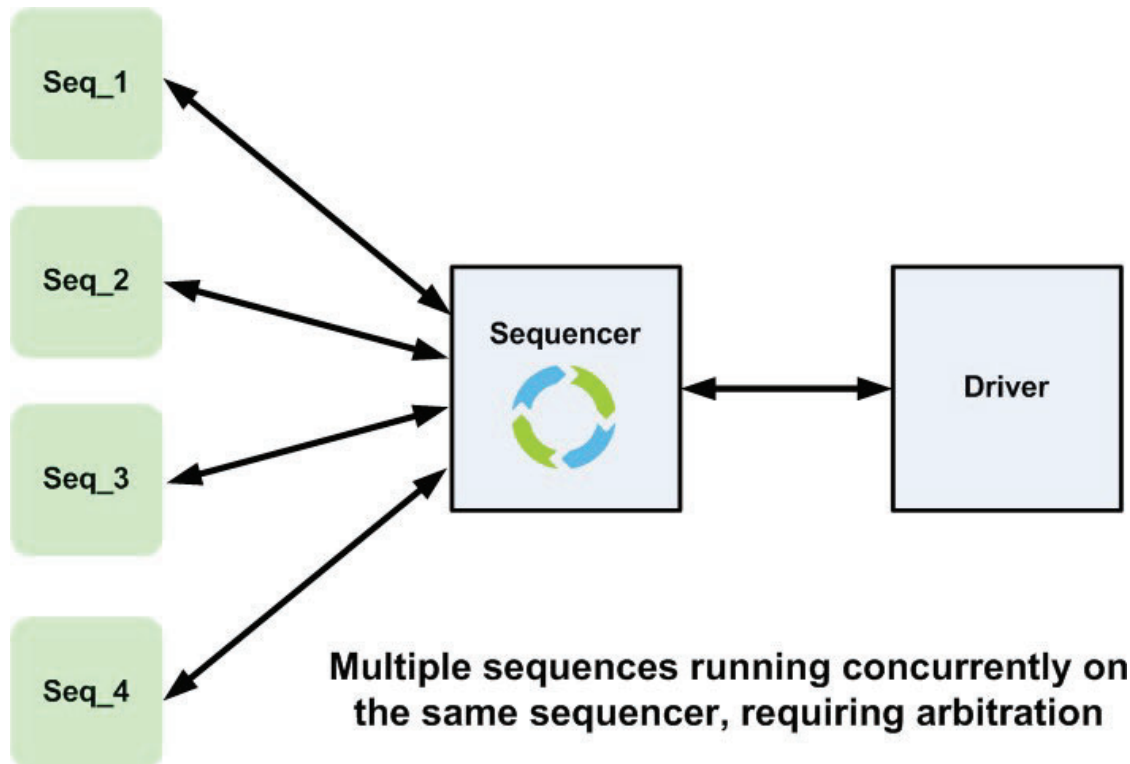
```
req.MWDATA, req.MADDR), UVM_MEDIUM);
end

foreach (addr[i]) begin
    start_item(req);
    req.MADDR = addr[i];
    req.MREAD = 1;
    finish_item(req);
    `uvm_info("", $sformatf("read (i = %0d) of %h at %h", i,
req.MRDATA, req.MADDR), UVM_MEDIUM);
end
endtask: body

endclass: mbus_pipelined_seq
```

Arbitrating Between Sequences

The uvm_sequencer has a built-in mechanism to arbitrate between sequences which could be running concurrently on a sequencer. The arbitration algorithm determines which sequence is granted access to send its sequence_item to the driver. There is a choice of six arbitration algorithms which can be selected using the set_arbitration() sequencer method from the controlling sequence.



Consider the example illustrated in the diagram. In this example we have four sequences which are running as sub-sequences of the main sequence. Each sequence sends sequence_items to the driver with its own id, the driver keeps count of how many of which ids have been received. The sequences have different priorities - seq_1 and seq_2 have the highest priority, then seq_3, with seq_4 having the lowest priority. The sequences generate sequence_items at different time offsets, with seq_1 starting first, followed by seq_2 and so on.

The master sequence generation loop is shown in this code snippet:

```
task body;
    seq_1 = arb_seq::type_id::create("seq_1"); seq_1.seq_no = 1;
    seq_2 = arb_seq::type_id::create("seq_2"); seq_2.seq_no = 2;
    seq_3 = arb_seq::type_id::create("seq_3"); seq_3.seq_no = 3;
    seq_4 = arb_seq::type_id::create("seq_4"); seq_4.seq_no = 4;

    m_sequencer.set_arbitration(arb_type); // arb_type is set by the test
    fork
        begin
            repeat(4) begin
                #1; // Offset by 1
                seq_1.start(m_sequencer, this, 500); // Highest priority
            end
        end
        begin
            repeat(4) begin
                #2; // Offset by 2
                seq_2.start(m_sequencer, this, 500); // Highest priority
            end
        end
        begin
            repeat(4) begin
                #3; // Offset by 3
                seq_3.start(m_sequencer, this, 300); // Medium priority
            end
        end
        begin
            repeat(4) begin
                #4; // Offset by 4
                seq_4.start(m_sequencer, this, 200); // Lowest priority
            end
        end
    join
endtask: body
```

The six sequencer arbitration algorithms are best understood in the context of this example.

NOTE:

From UVM1.2 onwards, including UVM 1800.2, the names of the enum elements were changed to add the "UVM_" prefix.

For instance UVM_SEQ_ARB_FIFO was known as SEQ_ARB_FIFO in UVM versions prior to UVM1.2.

UVM_SEQ_ARB_FIFO (Default)

This is the default sequencer arbitration algorithm. What it does is to send sequence_items from the sequencer to the driver in the order they are received by the sequencer, regardless of any priority that has been set. In the example, this means that the driver receives sequence items in turn from seq_1, seq_2, seq_3 and seq_4. The resultant log file is as follows:

```
# UVM_INFO @ 0: [Sqr Arb selected:] UVM_SEQ_ARB_FIFO
# UVM_INFO @ 1: [RCVD] Totals: SEQ_1:1 SEQ_2:0 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 11: [RCVD] Totals: SEQ_1:1 SEQ_2:1 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 21: [RCVD] Totals: SEQ_1:2 SEQ_2:1 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 31: [RCVD] Totals: SEQ_1:2 SEQ_2:1 SEQ_3:1 SEQ_4:0
# UVM_INFO @ 41: [RCVD] Totals: SEQ_1:2 SEQ_2:1 SEQ_3:1 SEQ_4:1
# UVM_INFO @ 51: [RCVD] Totals: SEQ_1:2 SEQ_2:2 SEQ_3:1 SEQ_4:1
# UVM_INFO @ 61: [RCVD] Totals: SEQ_1:3 SEQ_2:2 SEQ_3:1 SEQ_4:1
# UVM_INFO @ 71: [RCVD] Totals: SEQ_1:3 SEQ_2:2 SEQ_3:2 SEQ_4:1
# UVM_INFO @ 81: [RCVD] Totals: SEQ_1:3 SEQ_2:2 SEQ_3:2 SEQ_4:2
# UVM_INFO @ 91: [RCVD] Totals: SEQ_1:3 SEQ_2:3 SEQ_3:2 SEQ_4:2
# UVM_INFO @ 101: [RCVD] Totals: SEQ_1:4 SEQ_2:3 SEQ_3:2 SEQ_4:2
# UVM_INFO @ 111: [RCVD] Totals: SEQ_1:4 SEQ_2:3 SEQ_3:3 SEQ_4:2
# UVM_INFO @ 121: [RCVD] Totals: SEQ_1:4 SEQ_2:3 SEQ_3:3 SEQ_4:3
# UVM_INFO @ 131: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:3 SEQ_4:3
# UVM_INFO @ 141: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:4 SEQ_4:3
# UVM_INFO @ 151: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:4 SEQ_4:4
```

UVM_SEQ_ARB_WEIGHTED

With this algorithm, the sequence_items to be sent to the driver are selected on a random basis but weighted with the sequence_items from the highest priority sequence being sent first. In the example, this means that sequence_items from seq_1 and seq_2 are selected more often than ones from lower priority seq_3 and seq_4. The resultant log file illustrates this:

```
# UVM_INFO @ 0: [Sqr Arb selected:] UVM_SEQ_ARB_WEIGHTED
# UVM_INFO @ 1: [RCVD] Totals: SEQ_1:1 SEQ_2:0 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 11: [RCVD] Totals: SEQ_1:1 SEQ_2:0 SEQ_3:0 SEQ_4:1
# UVM_INFO @ 21: [RCVD] Totals: SEQ_1:1 SEQ_2:1 SEQ_3:0 SEQ_4:1
# UVM_INFO @ 31: [RCVD] Totals: SEQ_1:1 SEQ_2:2 SEQ_3:0 SEQ_4:1
# UVM_INFO @ 41: [RCVD] Totals: SEQ_1:1 SEQ_2:3 SEQ_3:0 SEQ_4:1
# UVM_INFO @ 51: [RCVD] Totals: SEQ_1:1 SEQ_2:3 SEQ_3:1 SEQ_4:1
# UVM_INFO @ 61: [RCVD] Totals: SEQ_1:1 SEQ_2:4 SEQ_3:1 SEQ_4:1
# UVM_INFO @ 71: [RCVD] Totals: SEQ_1:2 SEQ_2:4 SEQ_3:1 SEQ_4:1
# UVM_INFO @ 81: [RCVD] Totals: SEQ_1:3 SEQ_2:4 SEQ_3:1 SEQ_4:1
# UVM_INFO @ 91: [RCVD] Totals: SEQ_1:3 SEQ_2:4 SEQ_3:1 SEQ_4:2
# UVM_INFO @ 101: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:1 SEQ_4:2
# UVM_INFO @ 111: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:2 SEQ_4:2
# UVM_INFO @ 121: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:3 SEQ_4:2
# UVM_INFO @ 131: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:3 SEQ_4:3
# UVM_INFO @ 141: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:4 SEQ_4:3
# UVM_INFO @ 151: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:4 SEQ_4:4
```

UVM_SEQ_ARB_RANDOM

With this algorithm, the sequence_items are selected on a random basis, irrespective of the priority level of their controlling sequences. The result in the example is that sequence_items are sent to the driver in a random order irrespective of their arrival time at the sequencer and of the priority of the sequencer that sent them:

```
# UVM_INFO @ 0: [Sqr Arb selected:] UVM_SEQ_ARB_RANDOM
# UVM_INFO @ 1: [RCVD] Totals: SEQ_1:1 SEQ_2:0 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 11: [RCVD] Totals: SEQ_1:1 SEQ_2:0 SEQ_3:0 SEQ_4:1
# UVM_INFO @ 21: [RCVD] Totals: SEQ_1:1 SEQ_2:0 SEQ_3:0 SEQ_4:2
# UVM_INFO @ 31: [RCVD] Totals: SEQ_1:1 SEQ_2:1 SEQ_3:0 SEQ_4:2
# UVM_INFO @ 41: [RCVD] Totals: SEQ_1:1 SEQ_2:1 SEQ_3:1 SEQ_4:2
# UVM_INFO @ 51: [RCVD] Totals: SEQ_1:1 SEQ_2:2 SEQ_3:1 SEQ_4:2
# UVM_INFO @ 61: [RCVD] Totals: SEQ_1:2 SEQ_2:2 SEQ_3:1 SEQ_4:2
# UVM_INFO @ 71: [RCVD] Totals: SEQ_1:3 SEQ_2:2 SEQ_3:1 SEQ_4:2
# UVM_INFO @ 81: [RCVD] Totals: SEQ_1:3 SEQ_2:2 SEQ_3:1 SEQ_4:3
# UVM_INFO @ 91: [RCVD] Totals: SEQ_1:3 SEQ_2:3 SEQ_3:1 SEQ_4:3
# UVM_INFO @ 101: [RCVD] Totals: SEQ_1:3 SEQ_2:4 SEQ_3:1 SEQ_4:3
# UVM_INFO @ 111: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:1 SEQ_4:3
# UVM_INFO @ 121: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:2 SEQ_4:3
# UVM_INFO @ 131: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:2 SEQ_4:4
# UVM_INFO @ 141: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:3 SEQ_4:4
# UVM_INFO @ 151: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:4 SEQ_4:4
```

UVM_SEQ_ARB_STRICT_FIFO

The UVM_SEQ_ARB_STRICT_FIFO algorithm sends sequence_items to the driver based on their priority and their order in the FIFO, with highest priority items being sent in the order received. The result in the example is that the seq_1 and seq_2 sequence_items are sent first interleaved with each other according to the order of their arrival in the sequencer queue, followed by seq_3s sequence items, and then the sequence_items for seq_4.

```
# UVM_INFO @ 0: [Sequencer Arbitration selected:]
UVM_SEQ_ARB_STRICT_FIFO
# UVM_INFO @ 1: [RCVD] Totals: SEQ_1:1 SEQ_2:0 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 11: [RCVD] Totals: SEQ_1:1 SEQ_2:1 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 21: [RCVD] Totals: SEQ_1:2 SEQ_2:1 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 31: [RCVD] Totals: SEQ_1:2 SEQ_2:2 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 41: [RCVD] Totals: SEQ_1:3 SEQ_2:2 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 51: [RCVD] Totals: SEQ_1:3 SEQ_2:3 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 61: [RCVD] Totals: SEQ_1:4 SEQ_2:3 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 71: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 81: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:1 SEQ_4:0
# UVM_INFO @ 91: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:2 SEQ_4:0
# UVM_INFO @ 101: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:3 SEQ_4:0
# UVM_INFO @ 111: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:4 SEQ_4:0
# UVM_INFO @ 121: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:4 SEQ_4:1
# UVM_INFO @ 131: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:4 SEQ_4:2
```

```
# UVM_INFO @ 141: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:4 SEQ_4:3
# UVM_INFO @ 151: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:4 SEQ_4:4
```

UVM_SEQ_ARB_STRICT_RANDOM

This algorithm selects the sequence_items to be sent in a random order but weighted by the priority of the sequences which are sending them. The effect in the example is that seq_1 is selected randomly first and its sequence_items are sent before the items from seq_2, followed by seq_3 and then seq_4

```
# UVM_INFO @ 0: [Sqr Arb selected:] UVM_SEQ_ARB_STRICT_RANDOM
# UVM_INFO @ 1: [RCVD] Totals: SEQ_1:1 SEQ_2:0 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 11: [RCVD] Totals: SEQ_1:2 SEQ_2:0 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 21: [RCVD] Totals: SEQ_1:3 SEQ_2:0 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 31: [RCVD] Totals: SEQ_1:3 SEQ_2:1 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 41: [RCVD] Totals: SEQ_1:3 SEQ_2:2 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 51: [RCVD] Totals: SEQ_1:4 SEQ_2:2 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 61: [RCVD] Totals: SEQ_1:4 SEQ_2:3 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 71: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 81: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:1 SEQ_4:0
# UVM_INFO @ 91: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:2 SEQ_4:0
# UVM_INFO @ 101: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:3 SEQ_4:0
# UVM_INFO @ 111: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:4 SEQ_4:0
# UVM_INFO @ 121: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:4 SEQ_4:1
# UVM_INFO @ 131: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:4 SEQ_4:2
# UVM_INFO @ 141: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:4 SEQ_4:3
# UVM_INFO @ 151: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:4 SEQ_4:4
```

UVM_SEQ_ARB_USER

This algorithm allows a user defined method to be used for arbitration. In order to do this, the uvm_sequencer must be extended to override the user_priority_arbitration() method. The method receives an argument which is the sequencers queue of sequence_items, the user implemented algorithm needs to return an integer to select one of the sequence_items from the queue. The method is able to call on other methods implemented in the sequencer base class to establish the properties of each of the sequences in the queue. For instance, the priority of each sequence item can be established using the get_seq_item_priority() call as illustrated in the following example:

```
//
// Return the item with the mean average priority
//
function int user_priority_arbitration(int avail_sequences[$]);
    int priority[] = new[avail_sequences.size]
    int sum = 0;
    bit mean_found = 0;

    for (i = 0; i < avail_sequences.size(); i++) begin
        priority[i] =
get_seq_item_priority(arb_sequence_q[avail_sequences[i]]);
        sum = sum + priority[i];
```

```

end
// take the mean priority
sum = sum/avail_sequences.size();
// Find the first sequence that matches this priority
foreach(priority[i]) begin
    if(priority[i] == sum) begin
        return avail_sequences[i];
    end
end
end
// Otherwise return the mode average:
return avail_sequences[(avail_sequences.size()/2)];

endfunction: user_priority_arbitration

```

In the following example, the `user_priority_arbitration` method has been modified to always select the last sequence_item that was received, this is more or less the inverse of the default arbitration mechanism.

```

class seq_arb_sequencer extends uvm_sequencer #(seq_arb_item);

`uvm_component_utils(seq_arb_sequencer)

function new(string name = "seq_arb_sequencer", uvm_component parent =
null);
    super.new(name, parent);
endfunction

// This method overrides the default user method
// It returns the last item in the sequence queue
function integer user_priority_arbitration(integer avail_sequences[$]);
    int end_index;
    end_index = avail_sequences.size() - 1;
    return (avail_sequences[end_index]);
endfunction // user_priority_arbitration

endclass: seq_arb_sequencer

```

In the example, using this algorithm has the effect of sending the sequence_items from seq_4, followed by seq_3, seq_2 and then seq_1.

```

# UVM_INFO @ 0: [Sqr Arb selected:] UVM_SEQ_ARB_USER
# UVM_INFO @ 1: [RCVD] Totals: SEQ_1:1 SEQ_2:0 SEQ_3:0 SEQ_4:0
# UVM_INFO @ 11: [RCVD] Totals: SEQ_1:1 SEQ_2:0 SEQ_3:0 SEQ_4:1
# UVM_INFO @ 21: [RCVD] Totals: SEQ_1:1 SEQ_2:0 SEQ_3:0 SEQ_4:2
# UVM_INFO @ 31: [RCVD] Totals: SEQ_1:1 SEQ_2:0 SEQ_3:0 SEQ_4:3
# UVM_INFO @ 41: [RCVD] Totals: SEQ_1:1 SEQ_2:0 SEQ_3:0 SEQ_4:4
# UVM_INFO @ 51: [RCVD] Totals: SEQ_1:1 SEQ_2:0 SEQ_3:1 SEQ_4:4
# UVM_INFO @ 61: [RCVD] Totals: SEQ_1:1 SEQ_2:0 SEQ_3:2 SEQ_4:4
# UVM_INFO @ 71: [RCVD] Totals: SEQ_1:1 SEQ_2:0 SEQ_3:3 SEQ_4:4
# UVM_INFO @ 81: [RCVD] Totals: SEQ_1:1 SEQ_2:0 SEQ_3:4 SEQ_4:4

```

```
# UVM_INFO @ 91: [RCVD] Totals: SEQ_1:2 SEQ_2:0 SEQ_3:4 SEQ_4:4
# UVM_INFO @ 101: [RCVD] Totals: SEQ_1:3 SEQ_2:0 SEQ_3:4 SEQ_4:4
# UVM_INFO @ 111: [RCVD] Totals: SEQ_1:4 SEQ_2:0 SEQ_3:4 SEQ_4:4
# UVM_INFO @ 121: [RCVD] Totals: SEQ_1:4 SEQ_2:1 SEQ_3:4 SEQ_4:4
# UVM_INFO @ 131: [RCVD] Totals: SEQ_1:4 SEQ_2:2 SEQ_3:4 SEQ_4:4
# UVM_INFO @ 141: [RCVD] Totals: SEQ_1:4 SEQ_2:3 SEQ_3:4 SEQ_4:4
# UVM_INFO @ 151: [RCVD] Totals: SEQ_1:4 SEQ_2:4 SEQ_3:4 SEQ_4:4
```

Sequence Priority

The UVM sequence use model allows multiple sequences to access a driver concurrently. The sequencer contains an arbitration mechanism that determines when a sequence_item from a sequence will be sent to a driver. When a sequence is started using the start() method one of the arguments that can be passed is an integer indicating the priority of that sequence. The higher the value of the integer, the higher the priority of the sequence. This priority can be used with the UVM_SEQ_ARB_WEIGHTED, UVM_SEQ_ARB_STRICT_FIFO, and UVM_SEQ_ARB_STRICT_RANDOM arbitration mechanisms, (and possibly the UVM_SEQ_ARB_USER algorithm, if it handles priority) to ensure that a sequence has the desired priority. **Note:** the remaining sequencer arbitration mechanisms do not take the sequence priority into account.

For instance, if a bus fabric master port has 3 sequences running on it to model software execution accesses (high priority), video data block transfer (medium priority) and irritant data transfer (low priority) then the hierarchical sequence controlling them would look like:

```
// Coding tip: Create an enumerated type to represent the different
priority levels
typedef enum {HIGH_PRIORITY = 500, MED_PRIORITY = 200, LOW_PRIORITY =
50} seq_priority_e;

// In the hierarchical sequence:

task body();
  op_codes = cpu_sw_seq::type_id::create("op_codes");
  video = video_data_seq::type_id::create("video");
  irritant = random_access_seq::type_id::create("irritant");

  m_sequencer.set_arbitration(UVM_SEQ_ARB_STRICT_FIFO); // Arbitration
mechanism that uses priority
  fork
    op_codes.start(m_sequencer, this, HIGH_PRIORITY);
    video.start(m_sequencer, this, MED_PRIORITY);
    irritant.start(m_sequencer, this, LOW_PRIORITY);
  join
endtask: body
```

When it is necessary to override the sequencer priority mechanism to model an interrupt or a high priority DMA transfer, then the sequencer locking mechanism should be used.

Locking or Grabbing a Sequencer

There are a number of modeling scenarios where one sequence needs to have exclusive access to a driver via a sequencer. One example of this type of scenario is a sequence which is responding to an interrupt. In order to accommodate this requirement, the uvm_sequencer has a locking mechanism which is implemented using two calls - lock() and grab(). In terms of modeling, a lock might be used to model a prioritized interrupt and a grab might be used to model a non-maskable interrupt (NMI). The lock() and grab() calls have antidote calls to release a lock and these are unlock() and ungrab().

lock

The sequencer lock method is called from a sequence and its effect is that the calling sequence will be granted exclusive access to the driver when it gets its next slot via the sequencer arbitration mechanism. Once lock is granted, no other sequences will be able to access the driver until the sequence issues an unlock() call which will then release the lock. The method is blocking and does not return until lock has been granted.

grab

The grab method is similar to the lock method, except that it takes immediate effect and will grab the next sequencer arbitration slot, overriding any sequence priorities in place. The only thing that stops a sequence from grabbing a sequencer is a pre-existing lock() or grab() condition.

unlock

The unlock sequencer function is called from within a sequence to give up its lock or grab. A locking sequence must call unlock before completion, otherwise the sequencer will remain locked.

ungrab

The ungrab function is an alias of unlock.

Related functions:

is_blocked

A sequence can determine if it is blocked by a sequencer lock condition by making this call. If it returns a 0, then the sequence is not blocked and will get a slot in the sequencer arbitration. However, the sequencer may get locked before the sequence has a chance to make a start_item() call.

is_grabbed

If a sequencer returns a 1 from this call, then it means that it has an active lock or grab in progress.

current_grabber

This function returns the handle of the sequence which is currently locking the sequencer. This handle could be used to stop the locking sequence or to call a function inside it to unlock it.

Gotchas:

When a hierarchical sequence locks a sequencer, then its child sequences will have access to the sequencer. If one of the child sequences issues a lock, then the parent sequence will not be able to start any parallel sequences or send

any sequence_items until the child sequence has unlocked.

A locking or grabbing sequence must always unlock before it completes, otherwise the sequencer will become deadlocked.

Example:

The following example has 4 sequences which run in parallel threads. One of the threads has a sequence that does a lock, and a sequence that does a grab, both are functionally equivalent with the exception of the lock or grab calls.

Locking sequence:

```
class lock_seq extends uvm_sequence #(seq_arb_item);

`uvm_object_utils(lock_seq)

int seq_no;

function new(string name = "lock_seq");
    super.new(name);
endfunction

task body();
    seq_arb_item REQ;

    if(m_sequencer.is_blocked(this)) begin
        uvm_report_info("lock_seq", "This sequence is blocked by an existing
lock");
    end else begin
        uvm_report_info("lock_seq", "This sequence is not blocked by an
existing lock");
    end

    // Lock call - which blocks until it is granted
    m_sequencer.lock(this);

    if(m_sequencer.is_grabbed()) begin
        if(m_sequencer.current_grabber() != this) begin
            uvm_report_info("lock_seq", "Lock sequence waiting for current grab
or lock to complete");
        end
    end

    REQ = seq_arb_item::type_id::create("REQ");
    REQ.seq_no = 6;
    repeat(4) begin
        start_item(REQ);
        finish_item(REQ);
    end

    // Unlock call - must be issued
```



```

    m_sequencer.unlock(this);
endtask: body

endclass: lock_seq

```

Grabbing sequence:

```

class grab_seq extends uvm_sequence #(seq_arb_item);

`uvm_object_utils(grab_seq)

function new(string name = "grab_seq");
    super.new(name);
endfunction

task body(); seq_arb_item
    REQ;

    if(m_sequencer.is_blocked(this)) begin
        uvm_report_info("grab_seq", "This sequence is blocked by an existing lock");
    end else begin
        uvm_report_info("grab_seq", "This sequence is not blocked by an existing lock");
    end

    // Grab call which blocks until grab has been granted
    m_sequencer.grab(this);

    if(m_sequencer.is_grabbed()) begin if(m_sequencer.current_grabber() !=
        this) begin
        uvm_report_info("grab_seq", "Grab sequence waiting for current grab or lock to complete");
    end
    end

    REQ = seq_arb_item::type_id::create("REQ"); REQ.seq_no = 5;
    repeat(4) begin
        start_item(REQ);
        finish_item(REQ);
    end

    // Ungrab which must be called to release the grab (lock)
    m_sequencer.ungrab(this);
endtask: body

endclass: grab_seq

```

The overall controlling sequence runs four sequences which send sequence_items to the driver with different levels of priority. The driver reports from which sequence it has received a sequence_item. The first grab_seq in the fourth thread jumps the arbitration queue. The lock_seq takes its turn and blocks the second grab_seq, which then executes immediately the lock_seq completes.

```
task body();
seq_1 = arb_seq::type_id::create("seq_1"); seq_1.seq_no = 1;
seq_2 = arb_seq::type_id::create("seq_2"); seq_2.seq_no = 2;
seq_3 = arb_seq::type_id::create("seq_3"); seq_3.seq_no = 3;
seq_4 = arb_seq::type_id::create("seq_4"); seq_4.seq_no = 4;
grab = grab_seq::type_id::create("grab"); lock =
lock_seq::type_id::create("lock");

m_sequencer.set_arbitration(arb_type);
fork begin // Thread 1
  repeat(10) begin
    #1;
    seq_1.start(m_sequencer, this, 500); // Highest priority
  end
end begin // Thread 2
  repeat(10) begin
    #2;
    seq_2.start(m_sequencer, this, 500); // Highest priority
  end
end begin // Thread 3
  repeat(10) begin
    #3;
    seq_3.start(m_sequencer, this, 300); // Medium priority
  end
end begin // Thread 4
  fork
    repeat(2) begin
      #4;
      seq_4.start(m_sequencer, this, 200); // Lowest priority
    end
    #10 grab.start(m_sequencer, this, 50);
  join
  repeat(1) begin
    #4 seq_4.start(m_sequencer, this, 200);
  end
  fork
    lock.start(m_sequencer, this, 200);
    #20 grab.start(m_sequencer, this, 50);
```

```

join
end join

endtask: body

```

The resultant simulation transcript is as follows:

(Note that there has been some editing of the transcript to ensure that it renders reasonably in pdf)

```

UVM_INFO @ 0: reporter [RNTST] Running test arb_test...
# UVM_INFO @ 0: [Sequencer Arbitration selected:] SEQ_ARB_FIFO
# UVM_INFO @ 1:      [RCVD] Type: 1 S1:1 S2:0 S3:0 S4:0 GB:0 LK:0
# UVM_INFO @ 10: [grab_seq] This seq is NOT blocked by an existing lock
# UVM_INFO @ 11:      [RCVD] Type: 5 S1:1 S2:0 S3:0 S4:0 GB:1 LK:0
# UVM_INFO @ 12: [is_blocked] This seq is blocked by a lock or a grab
# UVM_INFO @ 21:      [RCVD] Type: 5 S1:1 S2:0 S3:0 S4:0 GB:2 LK:0
# UVM_INFO @ 31:      [RCVD] Type: 5 S1:1 S2:0 S3:0 S4:0 GB:3 LK:0
# UVM_INFO @ 41:      [RCVD] Type: 5 S1:1 S2:0 S3:0 S4:0 GB:4 LK:0
# UVM_INFO @ 51:      [RCVD] Type: 2 S1:1 S2:1 S3:0 S4:0 GB:4 LK:0
# UVM_INFO @ 61:      [RCVD] Type: 3 S1:1 S2:1 S3:1 S4:0 GB:4 LK:0
# UVM_INFO @ 71:      [RCVD] Type: 4 S1:1 S2:1 S3:1 S4:1 GB:4 LK:0
# UVM_INFO @ 81:      [RCVD] Type: 1 S1:2 S2:1 S3:1 S4:1 GB:4 LK:0
# UVM_INFO @ 91:      [RCVD] Type: 2 S1:2 S2:2 S3:1 S4:1 GB:4 LK:0
# UVM_INFO @ 101:     [RCVD] Type: 3 S1:2 S2:2 S3:2 S4:1 GB:4 LK:0
# UVM_INFO @ 111:     [RCVD] Type: 4 S1:2 S2:2 S3:2 S4:2 GB:4 LK:0
# UVM_INFO @ 121:     [RCVD] Type: 1 S1:3 S2:2 S3:2 S4:2 GB:4 LK:0
# UVM_INFO @ 131:     [RCVD] Type: 2 S1:3 S2:3 S3:2 S4:2 GB:4 LK:0
# UVM_INFO @ 141:     [RCVD] Type: 3 S1:3 S2:3 S3:3 S4:2 GB:4 LK:0
# UVM_INFO @ 151:     [RCVD] Type: 4 S1:3 S2:3 S3:3 S4:3 GB:4 LK:0
# UVM_INFO @ 161: [lock_seq] This seq is NOT blocked by an existing lock
# UVM_INFO @ 161:     [RCVD] Type: 6 S1:3 S2:3 S3:3 S4:3 GB:4 LK:1
# UVM_INFO @ 171:     [RCVD] Type: 6 S1:3 S2:3 S3:3 S4:3 GB:4 LK:2
# UVM_INFO @ 181: [grab_seq] This seq IS BLOCKED by an existing lock
# UVM_INFO @ 181:     [RCVD] Type: 6 S1:3 S2:3 S3:3 S4:3 GB:4 LK:3
# UVM_INFO @ 191:     [RCVD] Type: 6 S1:3 S2:3 S3:3 S4:3 GB:4 LK:4
# UVM_INFO @ 201:     [RCVD] Type: 5 S1:3 S2:3 S3:3 S4:3 GB:5 LK:4
# UVM_INFO @ 211:     [RCVD] Type: 5 S1:3 S2:3 S3:3 S4:3 GB:6 LK:4
# UVM_INFO @ 221:     [RCVD] Type: 5 S1:3 S2:3 S3:3 S4:3 GB:7 LK:4
# UVM_INFO @ 231:     [RCVD] Type: 5 S1:3 S2:3 S3:3 S4:3 GB:8 LK:4
# UVM_INFO @ 241:     [RCVD] Type: 1 S1:4 S2:3 S3:3 S4:3 GB:8 LK:4
# UVM_INFO @ 251:     [RCVD] Type: 2 S1:4 S2:4 S3:3 S4:3 GB:8 LK:4
# UVM_INFO @ 261:     [RCVD] Type: 3 S1:4 S2:4 S3:4 S4:3 GB:8 LK:4
# UVM_INFO @ 271:     [RCVD] Type: 1 S1:5 S2:4 S3:4 S4:3 GB:8 LK:4
# UVM_INFO @ 281:     [RCVD] Type: 2 S1:5 S2:5 S3:4 S4:3 GB:8 LK:4
# UVM_INFO @ 291:     [RCVD] Type: 3 S1:5 S2:5 S3:5 S4:3 GB:8 LK:4
# UVM_INFO @ 301:     [RCVD] Type: 1 S1:6 S2:5 S3:5 S4:3 GB:8 LK:4
# UVM_INFO @ 311:     [RCVD] Type: 2 S1:6 S2:6 S3:5 S4:3 GB:8 LK:4
# UVM_INFO @ 321:     [RCVD] Type: 3 S1:6 S2:6 S3:6 S4:3 GB:8 LK:4
# UVM_INFO @ 331:     [RCVD] Type: 1 S1:7 S2:6 S3:6 S4:3 GB:8 LK:4

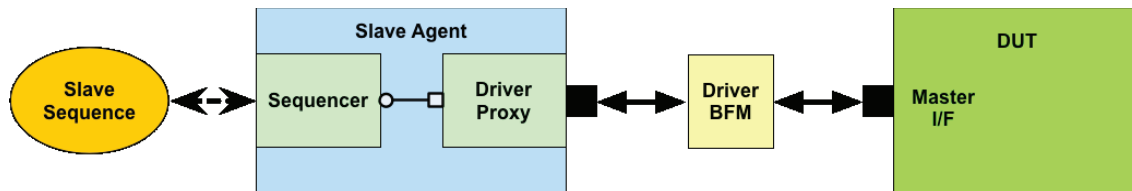
```

```
# UVM_INFO @ 341: [RCVD] Type: 2 S1:7 S2:7 S3:6 S4:3 GB:8 LK:4
# UVM_INFO @ 351: [RCVD] Type: 3 S1:7 S2:7 S3:7 S4:3 GB:8 LK:4
# UVM_INFO @ 361: [RCVD] Type: 1 S1:8 S2:7 S3:7 S4:3 GB:8 LK:4
# UVM_INFO @ 371: [RCVD] Type: 2 S1:8 S2:8 S3:7 S4:3 GB:8 LK:4
# UVM_INFO @ 381: [RCVD] Type: 3 S1:8 S2:8 S3:8 S4:3 GB:8 LK:4
# UVM_INFO @ 391: [RCVD] Type: 1 S1:9 S2:8 S3:8 S4:3 GB:8 LK:4
# UVM_INFO @ 401: [RCVD] Type: 2 S1:9 S2:9 S3:8 S4:3 GB:8 LK:4
# UVM_INFO @ 411: [RCVD] Type: 3 S1:9 S2:9 S3:9 S4:3 GB:8 LK:4
# UVM_INFO @ 421: [RCVD] Type: 1 S1:10 S2:9 S3:9 S4:3 GB:8 LK:4
# UVM_INFO @ 431: [RCVD] Type: 2 S1:10 S2:10 S3:9 S4:3 GB:8 LK:4
# UVM_INFO @ 441: [RCVD] Type: 3 S1:10 S2:10 S3:10 S4:3 GB:8 LK:4
```

Slave Sequences (Responders)

Overview

A slave sequence is used with a driver that responds to events on an interface rather than initiating them. This type of functionality is usually referred to as a responder.



A Slave Agent responds to requests on the master interface of the DUT

A responder can be implemented in several ways, for instance a simple bus oriented responder could be implemented as a `uvm_component` interacting with a slave interface and reading and writing from memory according to the requests from the bus master. The advantage of using a slave sequence is that the way in which the slave responds can be easily changed.

One interesting characteristic of responder functionality is that it is not usually possible to predict when a response to a request will be required. For this reason slave sequences tend to be implemented as long-lasting sequences, i.e. they last for the whole of the simulation providing the responder functionality.

In this article, two approaches to implementing a slave sequence are described:

- Using a single sequence item
- Using a sequence item for each slave phase (in the APB example used as an illustration there are two phases).

In both cases, the sequence and the driver loop through the following transitions:

1. Slave sequence sends a request to the driver - "Tell me what to do"
2. Driver detects a bus level request and returns the information back to the sequence - "This is what you should do"
3. Slave sequence does what it needs to do to prepare a response and then sends a response item to the driver - "Here you go"
4. Driver completes the bus level response with the contents of the response item, completes handshake back to the sequence - "Thank you"

Single Sequence Item Implementation

In this implementation, only one sequence item is used for both the request and response halves of the slave loop.

Sequence item

The response properties of a slave sequence item should be marked as rand and the properties driven during the master request should not. If you were to compare a master sequence_item and a slave sequence_item for the same bus protocol, then you would find that which properties were marked rand and which were not would be reversed.

```
class apb_slave_seq_item extends uvm_sequence_item;
  `uvm_object_utils(apb_slave_seq_item)

  //-----
  // Data Members (Outputs rand, inputs non-rand)
  //-----
  logic[31:0] addr;
  logic[31:0] wdata;
  logic rw;

  rand logic[31:0] rdata;
  rand logic slv_err;
  rand int delay;

  //-----
  // Constraints
  //-----
  constraint delay_bounds {
    delay inside {[0:2]};
  }

  constraint error_dist {
    slv_err dist {0 := 80, 1 := 20};
  }

  //-----
  // Methods
  //-----
  extern function new(string name = "apb_slave_seq_item");
  extern function void do_copy(uvm_object rhs);
  extern function bit do_compare(uvm_object rhs, uvm_comparer comparer);
  extern function string convert2string();
  extern function void do_print(uvm_printer printer);
  extern function void do_record(uvm_recorder recorder);

endclass: apb_slave_seq_item
```

Sequence and Driver

A slave device receives a request from the master and responds back with the relevant information. In order for the slave sequence to model this behavior, it needs to use multiple objects of the slave sequence item to accomplish the different phases. Depending on the bus protocol, at least two objects of the slave sequence item need to be used; one to get the request information and the second to return the response.

The process starts by the sequence sending a request object to the driver. The sequence does not need to initialize the contents of this object since it will be populated by the driver when it identifies a valid request on the pin level interface. When the driver calls the `item_done()` method, the slave sequence is unblocked and can use the contents of the request object to prepare the response object.

Once the slave sequence has processed the request, it creates and populates a response object and calls `start/finish_item()` to send it to the driver. The driver uses the response data to complete the response part of the pin level transfer and then unblocks the slave sequence execution by calling `item_done()`. The slave sequence may randomize the response data in some way.

Example slave sequence code:

```
task apb_slave_sequence::body;
    apb_slave_agent_config m_cfg =
apb_slave_agent_config::get_config(m_sequencer);
    apb_slave_seq_item req;
    apb_slave_seq_item rsp;

    m_cfg.wait_for_reset();
    forever begin

        req = apb_slave_seq_item::type_id::create("req");
        rsp = apb_slave_seq_item::type_id::create("rsp");

        // Slave request:
        start_item(req);
        finish_item(req);

        // Slave response:
        if (req.rw) begin
            memory[req.addr] = req.wdata;
        end
        start_item(rsp);
        rsp.copy(req);
        assert (rsp.randomize() with {if(!rsp.rw) rsp.rdata ==
memory[rsp.addr];});
        finish_item(rsp);
    end
endtask::body
```

Example slave driver code:

```
task apb_slave_driver::run_phase(uvm_phase phase);
    apb_slave_seq_item req;
```

```

apb_slave_seq_item rsp;

m_bfm.reset();

forever begin
    // Setup Phase
    seq_item_port.get_next_item(req);

    // Setup phase activity
    m_bfm.setup_phase(req);

    seq_item_port.item_done();

    // Access Phase
    seq_item_port.get_next_item(rsp);

    // Access phase activity
    m_bfm.access_phase(req, rsp);

    seq_item_port.item_done();
end

endtask: run_phase

```

Example	Download Link
Complete APB3 Slave Agent	

Multiple Sequence Items Implementation

Sequence items

In this implementation, we will use more than one sequence item (usually called phase level sequence items) to implement the slave functionality. Depending on the bus protocol, at least two sequence items will be required; one to implement the request phase and a second to implement the response phase. One way of looking at this is to consider it as the single sequence implementation sliced in two. However, with more complex protocols there could be more than two phase level sequence items.

The request sequence item will contain those data members that are not random and will, consequently, have no constraints. The response sequence item will contain all those random data members in addition to some data members that overlap with the request sequence item. Those overlapping members are needed by the response sequence item to make some decisions, for example a read/write bit is required by the driver to know if it needs to drive the read data bus with valid data.

For example, this is the APB3 slave setup (request) sequence item.

```

class apb_slave_setup_item extends apb_sequence_item;
    `uvm_object_utils(apb_slave_setup_item)

    //-----
    // Data Members (Outputs rand, inputs non-rand)

```

```
//-----
logic[31:0] addr;
logic[31:0] wdata;
logic rw;

endclass
```

And this is the access (response) sequence item. Note the rand data members and the constraints.

```
class apb_slave_access_item extends apb_sequence_item;
  `uvm_object_utils(apb_slave_setup_item)

  //-----
  // Data Members (Outputs rand, inputs non-rand)
  //-----
  rand logic rw;

  rand logic[31:0] rdata;
  rand logic slv_err;
  rand int delay;

  constraint delay_bounds {
    delay inside {[0:2]};
  }

  constraint error_dist {
    slv_err dist {0 := 80, 1 := 20};
  }
endclass
```

Sequence and Driver

The slave sequence and driver in this implementation are very similar to that described above. However, the major difference is that the sequencer and the driver are parameterized with the `apb_sequence_item` base class so that both request and response sequence item type objects can be passed between the sequence and the driver. The driver casts the received sequence item object to the appropriate request or response sequence item type before it accesses the contents and calls the BFM.

For the sequence, the only difference will be in the parameterization of the class template and no casting is required at all. The sequence body remains the same.

```
class apb_slave_sequence extends uvm_sequence #(apb_sequence_item);
```

As a consequence, the Sequencer's class definition will change as well.

```
class apb_slave_sequencer extends uvm_sequencer #(apb_sequence_item);
```

The driver's class definition will change too.

```
class apb_slave_driver extends uvm_driver #(apb_sequence_item,
apb_sequence_item);
```


The `run_phase` of the driver will always use the `uvm_sequence_item` to `get_next_item` and then cast the received sequence item to the appropriate/correct type.

```
task apb_slave_driver::run_phase(uvm_phase phase);
    apb_sequence_item item;
    apb_slave_setup_item req;
    apb_slave_access_item rsp;

    m_bfm.reset();

    forever begin
        // Setup Phase
        seq_item_port.get_next_item(item);
        if ($cast(req, item)) begin
            m_bfm.setup_phase(req);
        end else begin
            `uvm_error("CASTFAIL", "The received sequence item is not a
request seq_item");
        end
        seq_item_port.item_done();

        // Access Phase
        seq_item_port.get_next_item(item);
        if ($cast(rsp, item)) begin
            m_bfm.access_phase(rsp);
        end else begin
            `uvm_error("CASTFAIL", "The received sequence item is not a
response seq_item");
        end
        seq_item_port.item_done();
    end
endtask: run_phase
```

Example	Download Link
Complete APB3 Slave Agent using multiple sequence items	

Wait for a Signal

In the general case, synchronizing to hardware events is taken care of in UVM testbenches by drivers (proxy & BFM) and monitors (proxy & BFM). However, there are some cases where it is useful for a sequence or a component to synchronize to a hardware event such as a clock without interacting with a driver or a monitor. This can be facilitated by adding methods to an object containing a virtual interface BFM handle (usually a configuration object) that call a task in the BFM that blocks until a hardware event occurs. A further refinement is to add a `get_config()` method which allows a component to retrieve a pointer to its configuration object based on its scope within the UVM testbench component hierarchy.

BFM Methods

Since BFM's are the only items which are allowed to access pins directly, the BFM's have hardware synchronization methods added to them which can then be called by code in a testbench such as configuration objects. Examples of hardware synchronization methods include:

- `wait_for_clock()`
- `wait_for_reset()`
- `wait_for_interrupt()`
- `interrupt_cleared()`

These hardware synchronization methods are declared explicitly as automatic because static entities such as interfaces have implicitly static tasks. Since multiple threads/locations in a testbench may want to wait for a number of clock cycles or wait for a reset signal, we need to be able to invoke the tasks multiple times without overwriting the values used for each invocation.

In general the monitor BFM should be targeted for adding the extra methods since the monitor BFM would be present in both active and passive modes.

An example BFM:

```
interface bidirect_bus_monitor_bfm (bus_if BUS);

//
// Task: wait_for_clock
//
// This method waits for n clock cycles.
task automatic wait_for_clock( int n = 1 );
    repeat( n ) @( posedge BUS.clk );
endtask : wait_for_clock

//
// Task: wait_for_reset
//
// This method waits for the end of reset.
task automatic wait_for_reset();
    // Wait for reset to end
    @(posedge BUS.resetn);
endtask : wait_for_reset

//
```

```
// Task: wait_for_error
//
task automatic wait_for_error;
  @(posedge error);
endtask: wait_for_error

//
// Task: wait_for_no_error
//
task automatic wait_for_no_error;
  @(negedge error);
endtask: wait_for_no_error

endinterface : bidirect_bus_monitor_bfm
```

Additions to the configuration object

In order to support the use of a `wait_for_interface_signal` method, hardware synchronization methods have to be added to a configuration object. These hardware synchronization methods reference methods within the virtual interface BFM(s) which the configuration object contains handles for.

An example:

```
class bus_agent_config extends uvm_object;

  `uvm_object_utils(bus_agent_config)

  virtual bidirect_bus_monitor_bfm mon_bfm; // This is the virtual
  interface with the methods to wait on

  function new(string name = "bus_agent_config");
    super.new(name);
  endfunction

  //
  // Task: wait_for_clock
  //
  // This method waits for n clock cycles.
  task wait_for_clock( int n = 1 );
    mon_bfm.wait_for_clock(n);
  endtask

  //
  // Task: wait_for_reset
  //
  // This method waits for the end of reset.
  task wait_for_reset();
    mon_bfm.wait_for_reset();
  endtask : wait_for_reset
```

```
//
// Task: wait_for_error
//
task wait_for_error();
    mon_bfm.wait_for_error();
endtask : wait_for_error

//
// Task: wait_for_no_error
//
task wait_for_no_error();
    mon_bfm.wait_for_no_error();
endtask : wait_for_no_error

endclass: bus_agent_config
```

Using the wait_for_interface_signal method

In order to use the wait_for_xxx() method, the sequence or component must first ensure that it has a valid pointer to the configuration object. The pointer may already have been set up during construction or it may require the sequence or component to call the get_config() static method. Since sequences are not part of the UVM component hierarchy, they need to reference it via the m_sequencer pointer.

Once the local configuration object pointer is valid, the method can be called via the configuration object handle.

A sequence example:

```
class bus_seq extends uvm_sequence #(bus_seq_item);

    `uvm_object_utils(bus_seq)

    bus_seq_item req;
    bus_agent_config m_cfg;

    rand int limit = 40; // Controls the number of iterations

    function new(string name = "bus_seq");
        super.new(name);
    endfunction

    task body;
        int i = 5;
        req = bus_seq_item::type_id::create("req");
        // Get the configuration object
        if (m_cfg == null)
            if(!uvm_config_db #(bus_agent_config)::get(null, get_full_name(),
"config", m_cfg)) begin
                `uvm_error("BODY", "bus_agent_config not found")
            end
        end
```

```

repeat(limit)
begin
    start_item(req);
    // The address is constrained to be within the address of the
GPIO function
    // within the DUT, The result will be a request item for a read
or a write
    if(!req.randomize() with {addr inside
{[32'h0100_0000:32'h0100_001C]}})) begin
        `uvm_error("body", "randomization failed for req")
    end
    finish_item(req);
    // Wait for interface clocks:
    m_cfg.wait_for_clock(i);
    i++;
    // The req handle points to the object that the driver has
updated with response data
    uvm_report_info("seq_body", req.convert2string());
end
endtask: body

endclass: bus_seq

```

A component example:

```

//
// A coverage monitor that should ignore coverage collected during an
error condition:
//

class transfer_link_coverage_monitor extends uvm_subscriber
#(trans_link_item);

`uvm_component_utils(transfer_link_coverage_monitor)

T pkt;

// Error monitor bit
bit no_error;
// Configuration:
bus_agent_config m_cfg;

covergroup tlcm_1;
    HDR: coverpoint pkt.hdr;
    SPREAD: coverpoint pkt.payload {
        bins spread[] {[0:1023], [1024:8095], [8096:$]}
    }
endgroup

```

```

}
cross HDR, SPREAD;
endgroup: tlcm_1

function new(string name = "transfer_link_coverage_monitor", uvm_component_parent
= null);
super.new(name, parent); tlcm_1 =
new;
endfunction

// The purpose of the run method is to monitor the state of the error
// line
task run_phase( uvm_phase phase ); no_error = 0;
// Get the configuration
if (m_cfg == null)
    if(!uvm_config_db #(bus_agent_config)::get(this, "", "bus_agent_config",
m_cfg)) begin
        `uvm_error("run_phase", "Configuration error: unable to find bus_agent_config")
    end
m_cfg.wait_for_reset(); // Nothing valid until reset is over
no_error = 1;

forever begin
    m_cfg.wait_for_error(); // Blocks until an error occurs
    no_error = 0;
    m_cfg.wait_for_no_error(); // Blocks until the error is removed
end
endtask: run_phase

function write(T t); pkt = t;
if(no_error == 1) begin // Don't sample if there is an error
    tlcm_1.sample();
end
endfunction: write

endclass: transfer_link_coverage_monitor

```

Interrupt Sequences

In hardware terms, an Interrupt is an event which triggers a new thread of processing. This new thread can either take the place of the current execution thread, which is the case with an interrupt service routine or it can be used to wake up a sleeping process to initiate hardware activity. Either way, the interrupt is treated as a sideband signal or event which is not part of the main bus or control protocol.

In CPU based systems, interrupts are typically managed in hardware by interrupt controllers which can be configured to accept multiple interrupt request lines, to enable and disable interrupts, prioritise them and latch their current status. This means that a typical CPU only has one interrupt request line which comes from an interrupt controller and when the CPU responds to the interrupt it accesses the interrupt controller to determine the source and takes the appropriate action to clear it down. Typically, the role of the testbench is to verify the hardware implementation of the interrupt controller, however, there are circumstances where interrupt controller functionality has to be implemented in the testbench.

In some systems, an interrupt service route is re-entrant, meaning that if an ISR is in progress and a higher priority interrupt occurs, then the new interrupt triggers the execution of a new ISR thread which returns control to the first ISR on completion.

A stimulus generation flow can be adapted to take account of hardware interrupts in one of several ways:

- Exclusive execution of an ISR sequence
- Prioritized execution of an ISR sequence or sequences
- Hardware triggered sequences

Hardware Interrupt Detection

Each one of these approaches relies on an interrupt being detected via a transition on a virtual interface signal. The most convenient way to detect this signal transition is to add a `wait_for_hardware_event` task into the configuration object that contains a pointer to the virtual interface. The overall control sequence can then call a `wait_for_event` task as a sideband activity rather than use an agent to implement the interrupt interface.

Exclusive ISR Sequence

The simplest way to model interrupt handling is to trigger the execution of a sequence that uses the `grab()` method to get exclusive access to the target sequencer. This is a disruptive way to interrupt other stimulus generation that is taking place, but it does emulate what happens when an ISR is triggered on a CPU. The interrupt sequence cannot be interrupted itself, and must make an `ungrab()` call before it completes.

The interrupt monitor is usually implemented in a forked process running in a control or virtual sequence. The forked process waits for an interrupt, then starts the ISR sequence. When the ISR sequence ends, the loop starts again by waiting for the next interrupt.

```
//  
// Sequence runs a bus intensive sequence on one thread  
// which is interrupted by one of four interrupts  
//  
class int_test_seq extends uvm_sequence #(bus_seq_item);  
  
`uvm_object_utils(int_test_seq)  
  
function new (string name = "int_test_seq");
```

```

endfunction

task body;
  set_ints setup_ints; // Main activity on the bus interface
  isr ISR; // Interrupt service routine
  int_config i_cfg; // Config containing wait_for_IRQx tasks

  setup_ints = set_ints::type_id::create("setup_ints");
  ISR = isr::type_id::create("ISR");
  i_cfg = int_config::get_config(m_sequencer); // Get the config

  // Forked process - two levels of forking
  fork
    setup_ints.start(m_sequencer); // Main bus activity
  begin
    forever begin
      fork // Waiting for one or more of 4 interrupts
        i_cfg.wait_for_IRQ0();
        i_cfg.wait_for_IRQ1();
        i_cfg.wait_for_IRQ2();
        i_cfg.wait_for_IRQ3();
      join_any
      disable fork;
      ISR.start(m_sequencer); // Start the ISR
    end
  end
  join_any // At the end of the main bus activity sequence
  disable fork;

endtask: body

endclass: int_test_seq

```

Inside the ISR, the first action in the body method is the grab(), and the last action is ungrab(). If the ungrab() call was made earlier in the ISR sequence, then the main processing sequence would be able to resume sending sequence_items to the bus interface.

```

//
// Interrupt service routine
//
// Looks at the interrupt sources to determine what to do
//
class isr extends uvm_sequence #(bus_seq_item);

`uvm_object_utils(isr)

function new (string name = "isr");

```



```

endfunction

rand logic[31:0] addr;
rand logic[31:0] write_data; rand bit
read_not_write; rand int delay;

bit error;
logic[31:0] read_data;

task body; bus_seq_item
    req;

    m_sequencer.grab(this); // Grab => Immediate exclusive access to
    sequencer

    req = bus_seq_item::type_id::create("req");

    // Read from the GPO register to determine the cause of the interrupt assert (req.randomize() with {addr
    == 32'h0100_0000; read_not_write == 1;});
    start_item(req);
    finish_item(req);

    // Test the bits and reset if active
    //
    // Note that the order of the tests implements a priority structure
    //
    req.read_not_write = 0;
    if(req.read_data[0] == 1) begin
        `uvm_info("ISR:BODY", "IRQ0 detected", UVM_LOW)
        req.write_data[0] = 0;
        start_item(req);
        finish_item(req);
        `uvm_info("ISR:BODY", "IRQ0 cleared", UVM_LOW)
    end
    if(req.read_data[1] == 1) begin
        `uvm_info("ISR:BODY", "IRQ1 detected", UVM_LOW)
        req.write_data[1] = 0;
        start_item(req);
        finish_item(req);
        `uvm_info("ISR:BODY", "IRQ1 cleared", UVM_LOW)
    end
    if(req.read_data[2] == 1) begin
        `uvm_info("ISR:BODY", "IRQ2 detected", UVM_LOW)
        req.write_data[2] = 0;
        start_item(req);

```

```

    finish_item(req);
    `uvm_info("ISR:BODY", "IRQ2 cleared", UVM_LOW)
end
if(req.read_data[3] == 1) begin
    `uvm_info("ISR:BODY", "IRQ3 detected", UVM_LOW)
    req.write_data[3] = 0;
    start_item(req);
    finish_item(req);
    `uvm_info("ISR:BODY", "IRQ3 cleared", UVM_LOW)
end
start_item(req); // Take the interrupt line low
finish_item(req);

m_sequencer.ungrab(this); // Ungrab the sequencer, let other sequences
in

endtask: body

endclass: isr

```

Note that the way in which this ISR has been coded allows for a degree of prioritization since each IRQ source is tested in order from IRQ0 to IRQ3.

Prioritised ISR Sequence

A less disruptive approach to implementing interrupt handling using sequences is to use sequence prioritization. Here, the interrupt monitoring thread starts the ISR sequence with a priority that is higher than that of the main process. This has the potential to allow other sequences with a higher priority than the ISR to gain access to the sequencer. Note that in order to use sequence prioritisation, the sequencer arbitration mode needs to be set to SEQ_ARB_STRICT_FIFO, SEQ_ARB_STRICT_RAND or STRICT_ARB_WEIGHTED.

Prioritizing ISR sequences also enables modeling of prioritized ISRs, i.e. the ability to be able to interrupt an ISR with a higher priority ISR. However, since sequences are functor objects rather than simple sub-routines, multiple ISR sequences can be active on a sequencer, all that prioritization affects is their ability to send a sequence_item to the driver. Therefore, whatever processing is happening in an ISR will still continue even if a higher priority ISR "interrupts" it, which means that sequence_items from the first lower priority ISR could potentially get through to the driver.

The following code example demonstrates four ISR sequences which are started with different priorities, allowing a higher priority ISR to execute in preference to a lower priority ISR.

```

class int_test_seq extends uvm_sequence #(bus_seq_item);

    `uvm_object_utils(int_test_seq)

    function new (string name = "int_test_seq");
        super.new(name);
    endfunction

    task body;
        set_ints setup_ints; // Main sequence running on the bus
    endtask

```

```
isr ISR0, ISR1, ISR2, ISR3; // Interrupt service routines

int_config i_cfg;

setup_ints = set_ints::type_id::create("setup_ints");
// ISR0 is the highest priority
ISR0 = isr::type_id::create("ISR0"); ISR0.id =
"ISR0";
ISR0.i = 0;
// ISR1 is medium priority
ISR1 = isr::type_id::create("ISR1"); ISR1.id =
"ISR1";
ISR1.i = 1;
// ISR2 is medium priority
ISR2 = isr::type_id::create("ISR2"); ISR2.id =
"ISR2";
ISR2.i = 2;
// ISR3 is lowest priority
ISR3 = isr::type_id::create("ISR3"); ISR3.id =
"ISR3";
ISR3.i = 3;

i_cfg = int_config::get_config(m_sequencer);

// Set up sequencer to use priority based on FIFO order
m_sequencer.set_arbitration(SEQ_ARB_STRICT_FIFO);

// A main thread, plus one for each interrupt ISR
fork setup_ints.start(m_sequencer); forever begin
  // Highest priority
  i_cfg.wait_for_IRQ0(); ISR0.isr_no++;
  ISR0.start(m_sequencer, this, HIGH);
end
  forever begin // Medium priority
  i_cfg.wait_for_IRQ1(); ISR1.isr_no++;
  ISR1.start(m_sequencer, this, MED);
end
  forever begin // Medium priority
  i_cfg.wait_for_IRQ2(); ISR2.isr_no++;
  ISR2.start(m_sequencer, this, MED);
end
  forever begin // Lowest priority
  i_cfg.wait_for_IRQ3();
```

```

    ISR3.isr_no++;
    ISR3.start(m_sequencer, this, LOW);
end
join_any
disable fork;

endtask: body

endclass: int_test_seq

```

Interrupt Driven Sequence Execution

Some types of hardware systems use interrupts to schedule events or to control the order of processing. For instance, DSP based systems often consist of a chain of hardware acceleration blocks that take the contents of a memory block, perform a transform on it and write the results to another block of memory generating an interrupt when the transform is complete. Interrupts are used because the length of time to complete the transform is variable and it is not safe to start the next stage in the processing chain until the one before has completed. Each DSP hardware accelerator generates an interrupt when it is ready to process, or when it has completed a transform. This is a different situation from the previous examples, because the sequence execution is reliant on the interrupts coming in:

```

// DSP Control Sequence
//
// Each DSP processor in the chain generates an interrupt
// when it has completed processing
//
// The sequence then starts the next accelerator
//
class dsp_con_seq extends uvm_sequence #(dsp_con_seq_item);

`uvm_object_utils(dsp_con_seq)

dsp_con_config cfg;
dsp_con_seq_item req;

function new(string name = "dsp_con_seq");
    super.new(name);
endfunction

// Wait for the interrupts to fire
// then start up the next DSP block
task body;

    cfg = dsp_con_config::get_config(m_sequencer);

    req = dsp_con_seq_item::type_id::create("req");

    cfg.wait_for_reset;
    repeat(2) begin
        do_go(4'h1); // Start Accelerator 0
    end
endtask

```

```
cfg.wait_for_irq0; // Accelerator 0 complete
do_go(4'h2); // Start Accelerator 1
cfg.wait_for_irq1; // Accelerator 1 complete
do_go(4'h4); // Start Accelerator 2
cfg.wait_for_irq2; // Accelerator 2 complete
do_go(4'h8); // Start Accelerator 3
cfg.wait_for_irq3; // Accelerator 3 complete
end
cfg.wait_for_clock;

endtask: body

// Toggles the go or start bit
task do_go(bit[3:0] go);
    req.go = go;
    start_item(req);
    finish_item(req);
endtask

endclass:dsp_con_seq
```

Stopping a Sequence

Once started, sequences should not be stopped.

There are two methods available in the sequence and sequencer API that allow sequences to be killed. However, neither method checks that the driver is not currently processing any `sequence_items` and the result is that any `item_done()` or `put()` method called from the driver will either never reach the controlling sequence or will cause an UVM fatal error to occur because the sequences return pointer queue has been flushed.

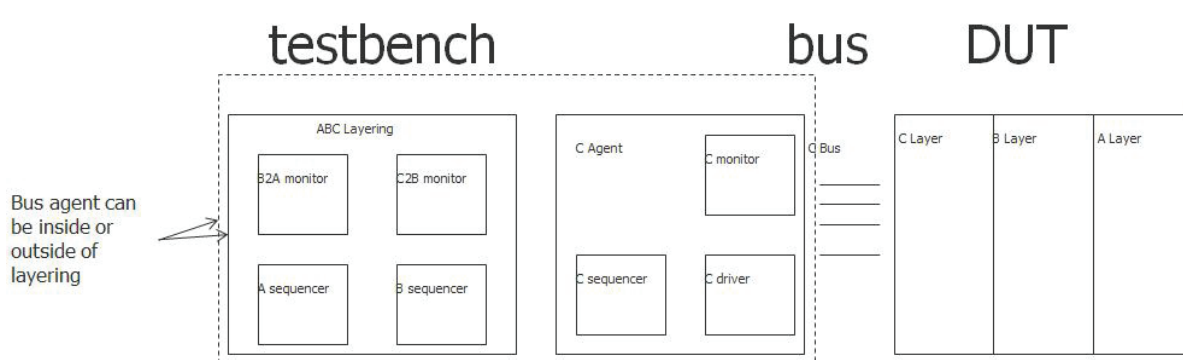
The methods are:

- `<sequence>.kill()`
- `<sequencer>.stop_sequences()`

Do not use these methods unless you have some way of ensuring that the driver is inactive before making the call.

Layering Sequences

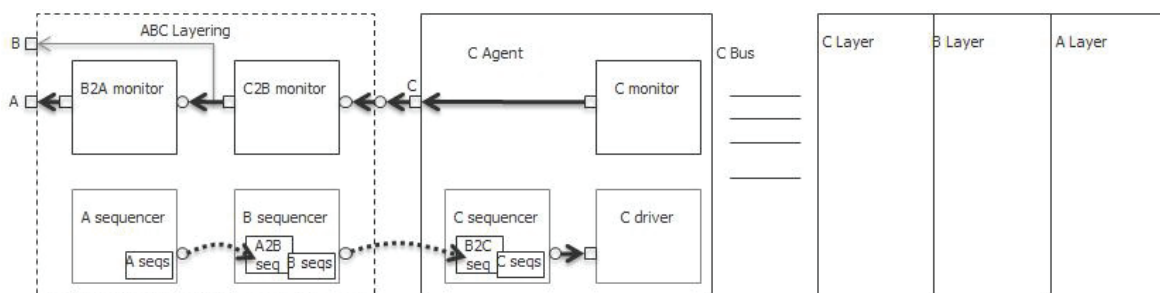
Many protocols have a hierarchical definition - for example, PCI express, USB 3.0, and MIPI LLI all have a Transaction Layer, a Transport Layer, and a Physical Layer. Sometimes we want to create a protocol independent layer on top of a standard protocol so that we can create protocol independent verification components (for example TLM 2.0 GP over AMBA AHB). All these cases require that we deconstruct sequence items of the higher level protocol into sequence items of the lower level protocol in order to stimulate the bus and that we reconstruct higher level sequence items from the lower level sequence items in the analysis datapath.



The Architecture of a Layering

In order to do this we construct a layering component. A layering component:

- Must include a child sequencer for each non leaf level in the layering.
- Must create, connect and start a translator sequence for each non leaf level in the layering.
- Must have a handle to the leaf level protocol agent. This protocol agent may be a child of the layering or external to it.
- May include a reconstruction monitor for each non leaf level in the layering.
- Should create and connect external analysis ports for each monitor contained within the layering
- Will usually have a configuration object associated with it that contains the configuration objects of all the components contained within it.



Data Flow

- ← = via formal port to export communication
- ←.. = via direct calls to upstream sequencer's export

Child Sequencers

A child sequencer in a layering is simply the usual sequencer for that protocol. Very often an appropriately parameterized `uvm_sequencer` is quite sufficient. If the higher level protocol has been modeled as a protocol UVC, then the layering should instantiate an instance of the sequencer used by the agent for that protocol so that sequences can be targeted either at the bus agent or the layering.

For example, the ABC layering below has an `A_sequencer` and a `B_sequencer`.

```
class ABC_layering extends uvm_subscriber #( C_item );
  `uvm_component_utils( ABC_layering )

  ...
  A_sequencer a_sequencer;
  B_sequencer b_sequencer;
  ...
  C_agent c_agent;

  function new(string name, uvm_component parent=null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    a_sequencer = A_sequencer::type_id::create("a_sequencer", this);
    b_sequencer = B_sequencer::type_id::create("b_sequencer", this);
  endfunction

  ...
endclass
```

Translator Sequences

A sequence which translates from upstream items to downstream items runs on the downstream sequencer but has a reference to the upstream sequencer. It directly references the upstream sequencer to call `get_next_item` and `item_done` to get upstream items and tell the upstream sequencer that it has finished processing the upstream sequence item. Between `get_next_item` and `item_done` it sends data to and gets data from the lower level sequencer by calling `start_item` and `finish_item`. A simple BtoC translator sequence is shown below:

```
class BtoC_seq extends uvm_sequence #(C_item);
  `uvm_object_utils(BtoC_seq);

  function new(string name="");
    super.new(name);
  endfunction

  uvm_sequencer #(B_item) up_sequencer;

  virtual task body();
    B_item b;
    C_item c;
    int i;
```

```

forever begin
  up_sequencer.get_next_item(b);
  foreach (b.fb[i]) begin
    c = C_item::type_id::create();

    start_item(c);
    c.fc = b.fb[i];
    finish_item(c);
  end
  up_sequencer.item_done();
end
endtask
endclass

```

The run phase of the ABC_layering component is responsible for creating the translator sequences, connecting them to their upstream sequencers, and starting them on their downstream sequencers:

```

virtual task run_phase(uvm_phase phase);
  AtoB_seq a2b_seq;
  BtoC_seq b2c_seq;

  a2b_seq = AtoB_seq::type_id::create("a2b_seq");
  b2c_seq = BtoC_seq::type_id::create("b2c_seq");

  // connect translation sequences to their respective upstream
sequencers
  a2b_seq.up_sequencer = a_sequencer;
  b2c_seq.up_sequencer = b_sequencer;

  // start the translation sequences
  fork
    a2b_seq.start(b_sequencer);
    b2c_seq.start(c_agent.c_sequencer);
  join_none
endtask

```

The Protocol Agent

Every layering must have a handle to the leaf level protocol agent. If we are delivering verification IP for a layered protocol, it usually makes sense to deliver the layering with an internal protocol agent. On the other hand, we may be adding a layering for use with a shrink wrapped protocol agent instantiated elsewhere in the testbench. Under these circumstances we will want the leaf level protocol agent to be outside the layering.

Internal Protocol Agent

In the case of an internal protocol agent, the layering component inherits from uvm_component and creates a child layering agent:

```

class ABC_layering extends uvm_component;
  `uvm_component_utils( ABC_layering )

```



```

...
A_sequencer a_sequencer;
B_sequencer b_sequencer;
...
C_agent c_agent;

function new(string name, uvm_component parent=null);
    super.new(name, parent);
endfunction

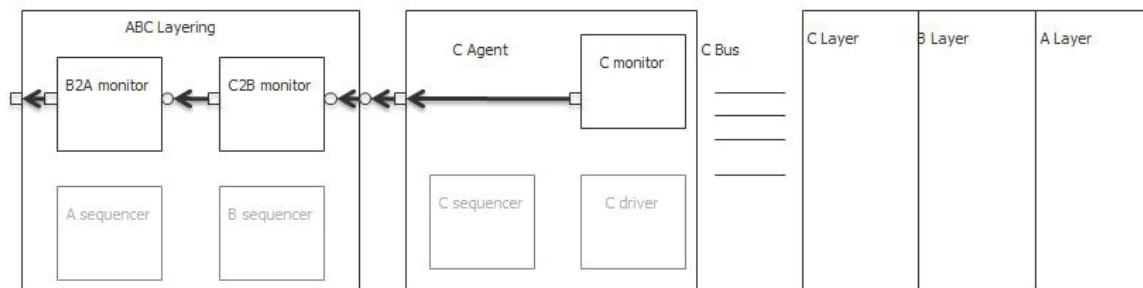
function void build_phase(uvm_phase phase);
    a_sequencer = A_sequencer::type_id::create("a_sequencer", this);
    b_sequencer = B_sequencer::type_id::create("b_sequencer", this);
    c_agent = C_agent::type_id::create("c_sequencer", this);
endfunction
...
endclass

```

External Protocol Agent

In the case of an external protocol agent, the layering is a subscriber parameterized on the leaf level sequence item and the agent is not constructed inside the layering. The code introduced above shows the code for an external agent.

The Analysis Path



Really, there is nothing special in the analysis path of a layering. For each layer in the monitoring we provide a reconstruction monitor which assembles high level items from low level items. These reconstruction monitors have an analysis export which is connected to the analysis ports of the lower level monitor and an analysis port. This analysis port is connected to an external analysis port and the analysis export of the upstream monitor if there is one.

An outline of a reconstruction monitor is shown below:

```

class C2B_monitor extends uvm_subscriber #(C_item); // provides an
analysis export of type C_item
    `uvm_component_utils(C2B_monitor)

    uvm_analysis_port#(B_item) ap;
    // declarations omitted ...

    function new(string name, uvm_component parent);

```

```

    super.new(name, parent);
    ap = new("ap", this);
endfunction

function void write(C_item t);
    // reconstruction code omitted ...
    ap.write( b_out );
    ...
endfunction
endclass

```

The reconstruction monitors are connected up in the normal way:

```

class ABC_layering extends uvm_subscriber #( C_item );
    `uvm_component_utils( ABC_layering )

    uvm_analysis_port #( A_item ) ap;

    A_sequencer a_sequencer;
    B_sequencer b_sequencer;

    C2B_monitor c2b_mon;
    B2A_monitor b2a_mon;

    C_agent c_agent;

    function new(string name, uvm_component parent=null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        a_sequencer = A_sequencer::type_id::create("a_sequencer", this);
        b_sequencer = B_sequencer::type_id::create("b_sequencer", this);

        c2b_mon = C2B_monitor::type_id::create("c2b_mon", this);
        b2a_mon = B2A_monitor::type_id::create("b2a_mon", this);

        ap = new("ap" , this );
    endfunction

    function void connect_phase(uvm_phase phase);
        c2b_mon.ap.connect(b2a_mon.analysis_export);
        b2a_mon.ap.connect( ap );
    endfunction

    ...
endclass

```